

Multiplexers

- A multiplexer routes (or connects) the selected data input to the output.
 - The value of the control inputs determines the data input that is selected.
- A multiplexer has
 - N control inputs
 - 2^N data inputs
 - 1 output

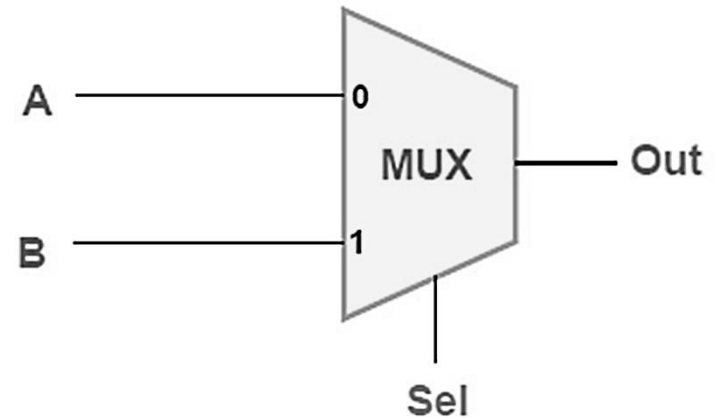
Multiplexer types

Multiplexer types are represented as **in : out** like 2:1 Mux

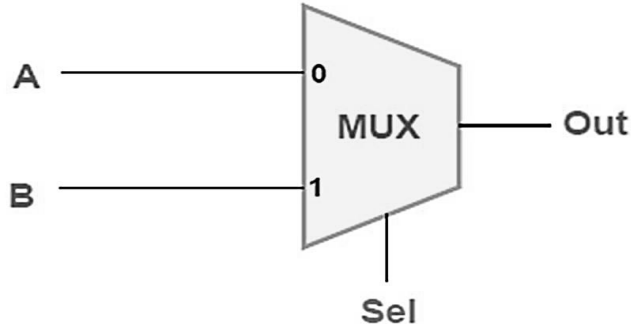
Selection signals (N)	Input signals (2^N)	Output signal (1)	MUX type
1	2	1	2:1 MUX
2	4	1	4:1 MUX
3	8	1	8:1 MUX
4	16	1	16:1 MUX
5	32	1	32:1 MUX

2:1 Multiplexer

- **Multiplexer:** Choose one of two inputs based on a control input
- **Sel:** Select line (it is a control input)
- **A, B:** Data Inputs
- **Out:** Multiplexer Output
- If Sel=0, Out=A
- If Sel=1, Out=B



2:1 MUX



- If *Sel* is '0', choose *A* and pass it on to *Out*
- If *Sel* is '1', choose *B* and pass it on to *Out*
- The value of *A* and *B* does not matter
 - Pass both 0's and 1's

Truth Table

<i>Sel</i>	<i>A</i>	<i>B</i>	<i>Out</i>
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

Characteristic Table

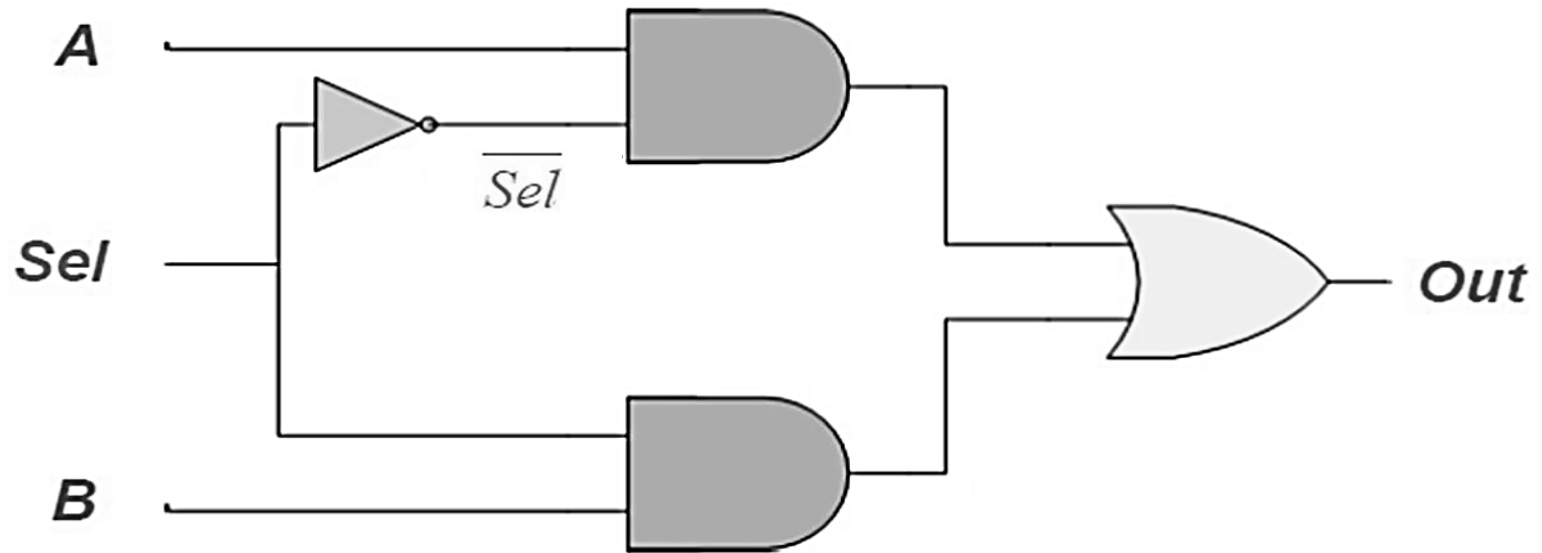
<i>Sel</i>	<i>Out</i>
0	<i>A</i>
1	<i>B</i>

Boolean Equation

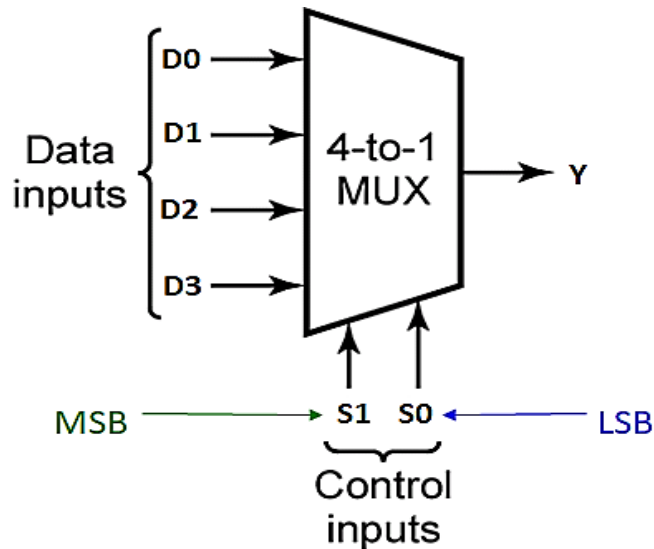
$$Out = \overline{Sel} \cdot A + Sel \cdot B$$

2:1 MUX

$$Out = \overline{Sel} \cdot A + Sel \cdot B$$

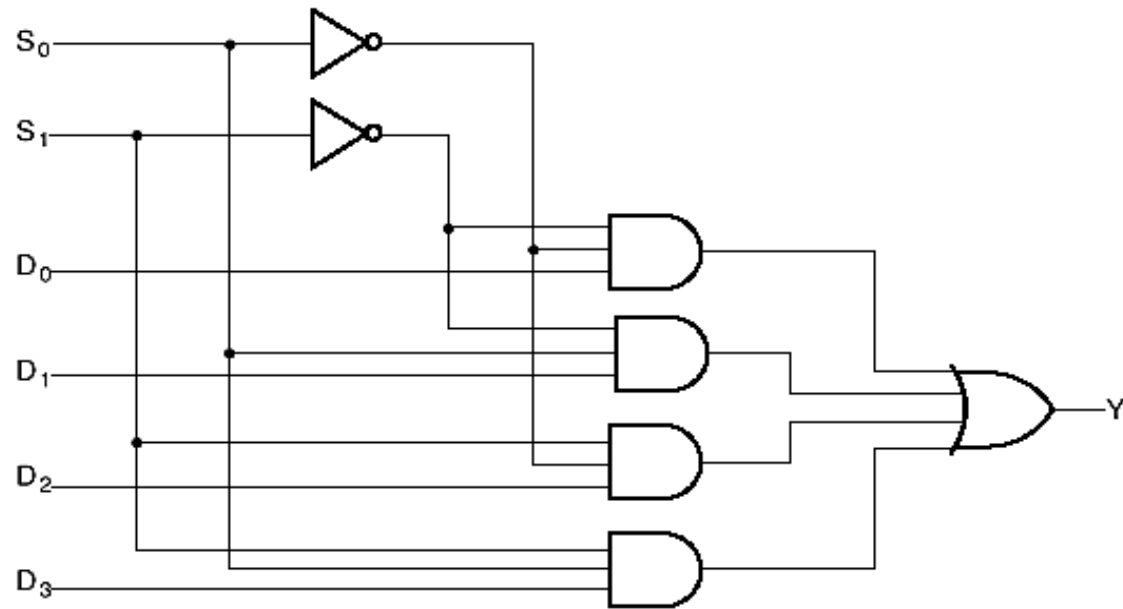


4:1 Multiplexer

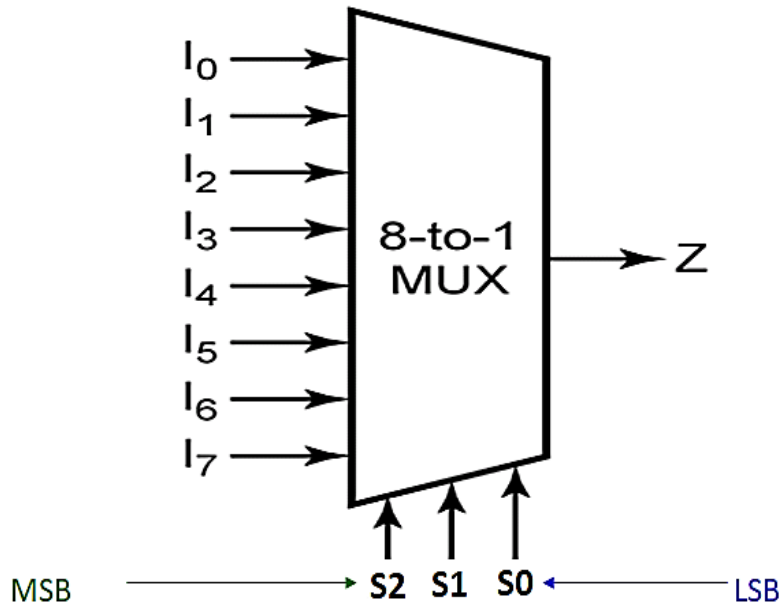


$$Y = S_1'S_0'D_0 + S_1'S_0D_1 + S_1S_0'D_2 + S_1S_0D_3$$

Function table		
S ₁	S ₀	Y
0	0	D ₀
0	1	D ₁
1	0	D ₂
1	1	D ₃



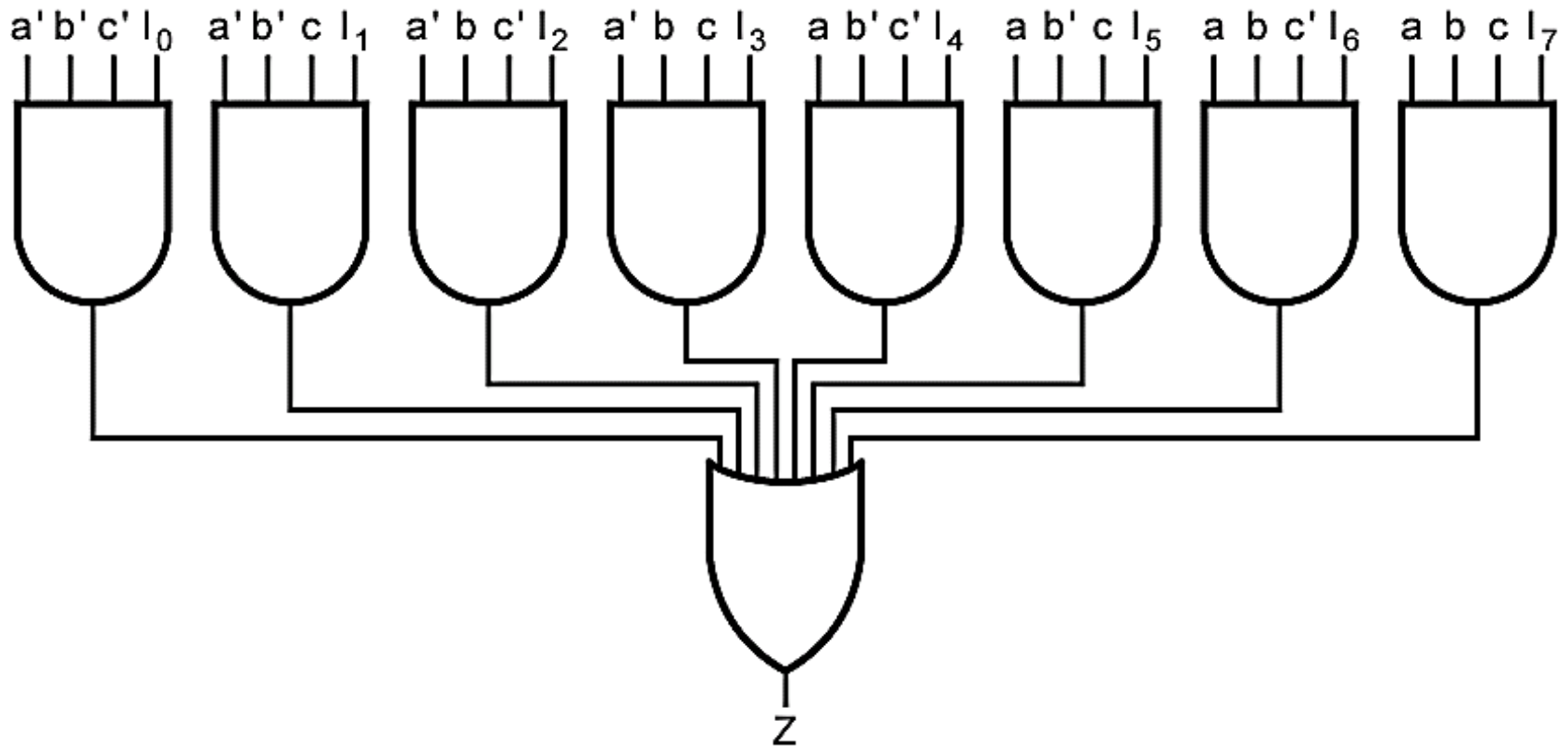
8:1 Multiplexer



S2	S1	S0	Z
0	0	0	I_0
0	0	1	I_1
0	1	0	I_2
0	1	1	I_3
1	0	0	I_4
1	0	1	I_5
1	1	0	I_6
1	1	1	I_7

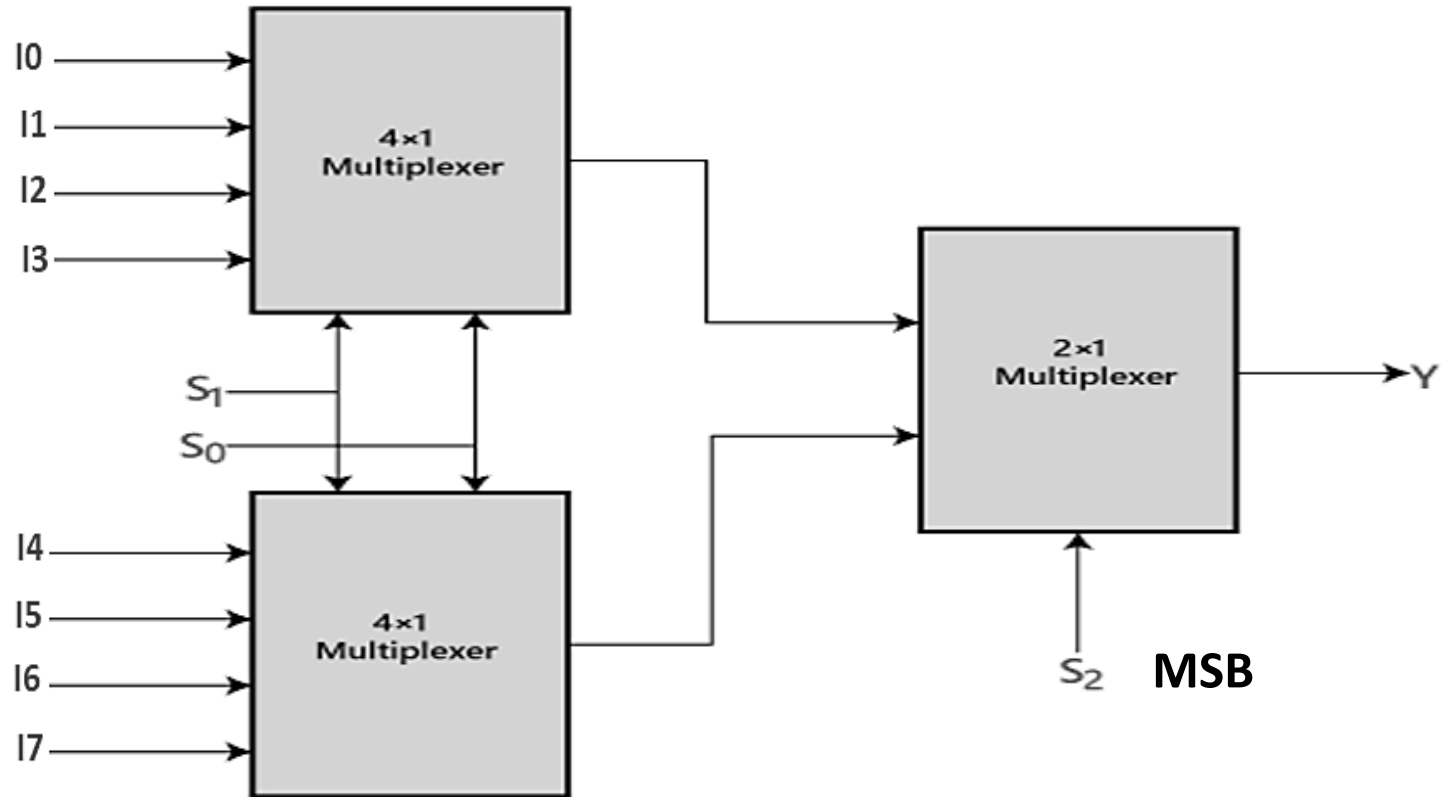
$$Z = S_2'.S_1'.S_0'.I_0 + S_2'.S_1'.S_0.I_1 + S_2'.S_1.S_0'.I_2 + S_2'.S_1.S_0.I_3 + \\ S_2.S_1'.S_0'.I_4 + S_2.S_1'.S_0.I_5 + S_2.S_1.S_0'.I_6 + S_2.S_1.S_0.I_7$$

8:1 Multiplexer



8:1 Mux design using 4:1 Mux and 2:1 Mux

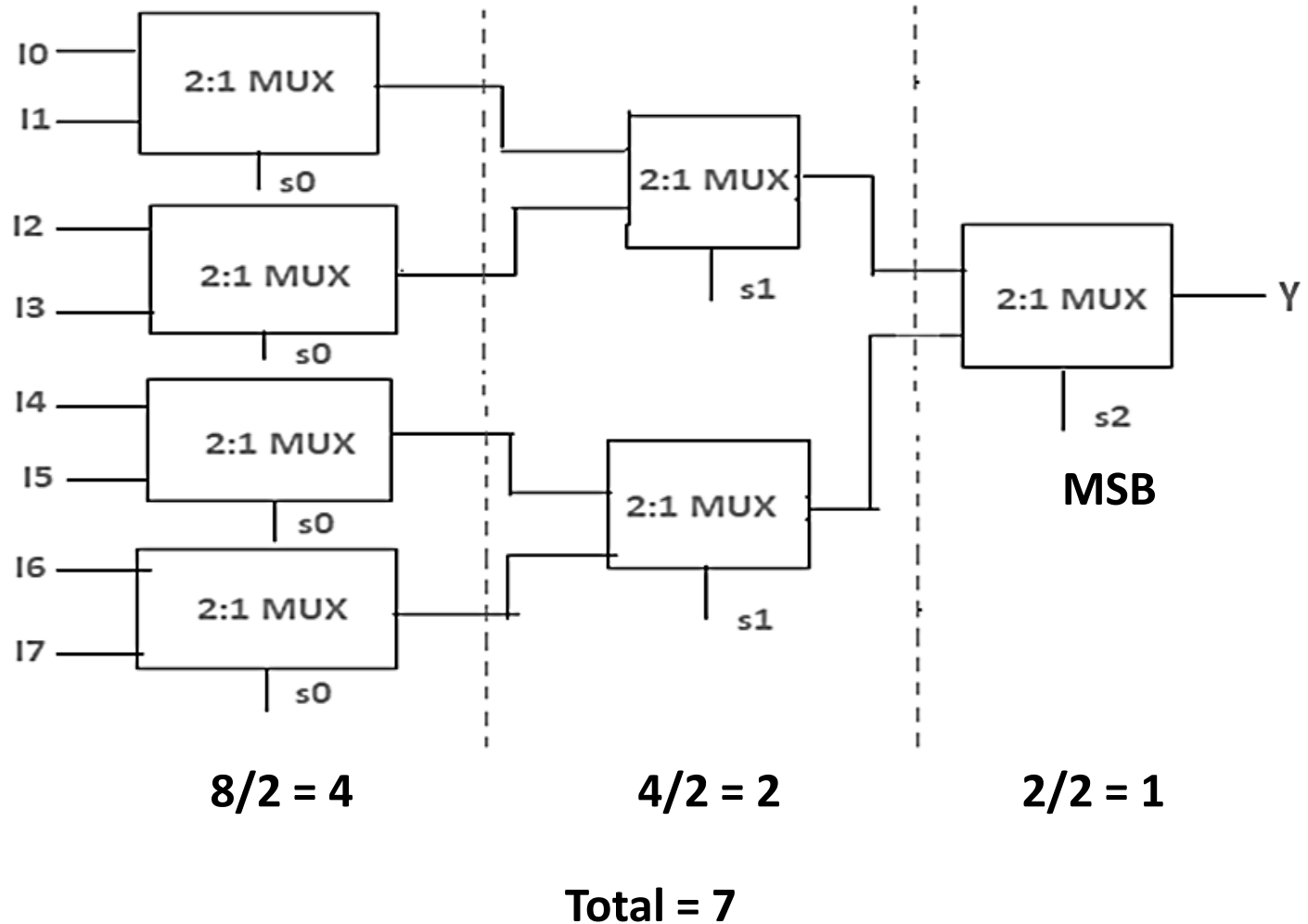
S2	S1	S0	Y
0	0	0	I ₀
0	0	1	I ₁
0	1	0	I ₂
0	1	1	I ₃
1	0	0	I ₄
1	0	1	I ₅
1	1	0	I ₆
1	1	1	I ₇



$$8/4 = 2$$

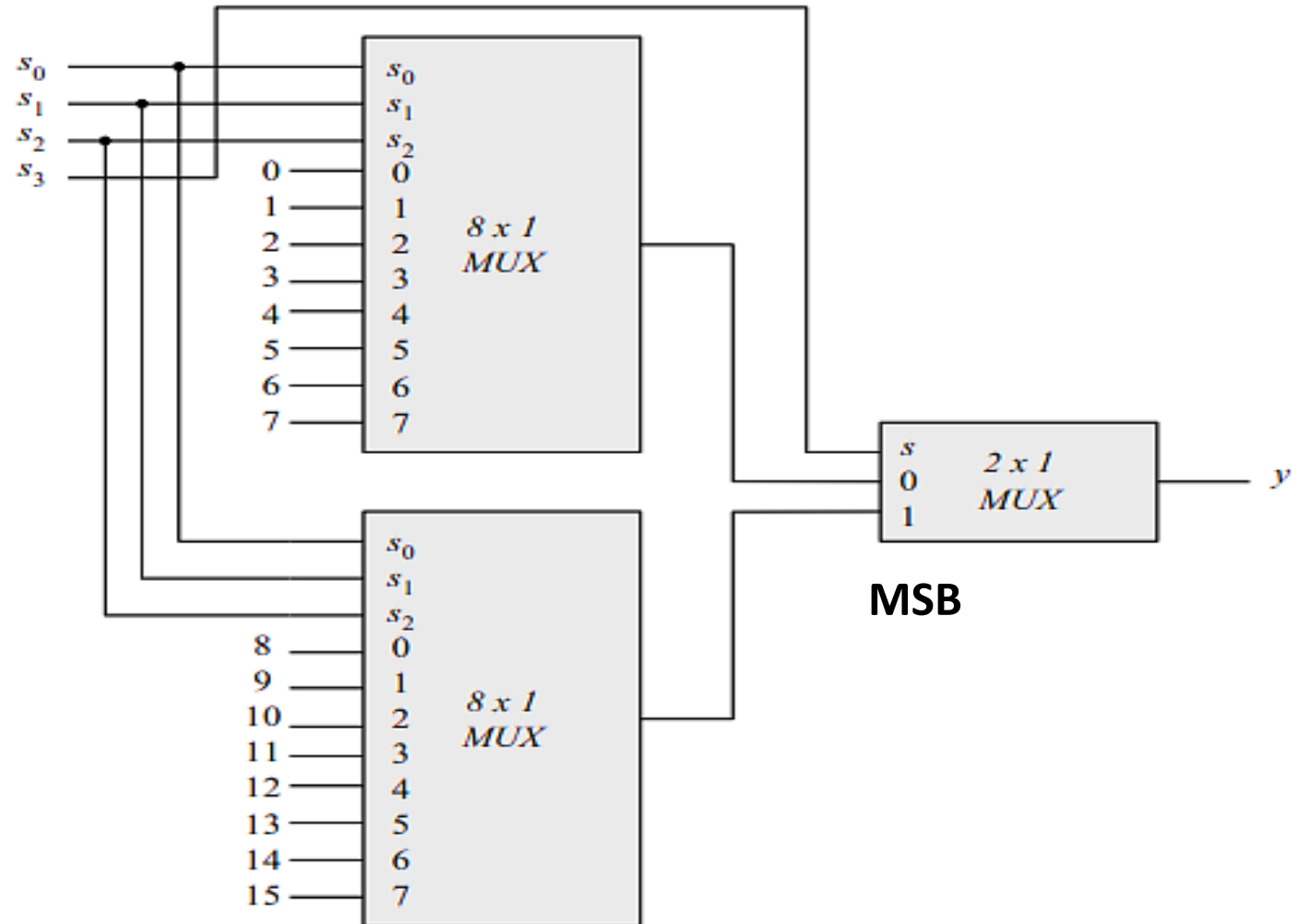
8:1 Mux design using 2:1 Mux

S2	S1	S0	Y
0	0	0	I ₀
0	0	1	I ₁
0	1	0	I ₂
0	1	1	I ₃
1	0	0	I ₄
1	0	1	I ₅
1	1	0	I ₆
1	1	1	I ₇



Problem: Construct a 16x1 multiplexer with two 8x1 and one 2x1 multiplexers. Use block diagrams

Solution:



Number of Mux

- 16:1 MUX requires how many 2:1 MUX?
 - $8+4+2+1=15$
- 64:1 MUX requires how many 2:1 MUX?
 - $32+16+8+4+2+1=63$
- 16:1 MUX requires how many 4:1 MUX?
 - $4+1=5$
- 64:1 MUX requires how many 4:1 MUX?
 - $16+4+1=21$

Boolean Function Implementation Using Mux

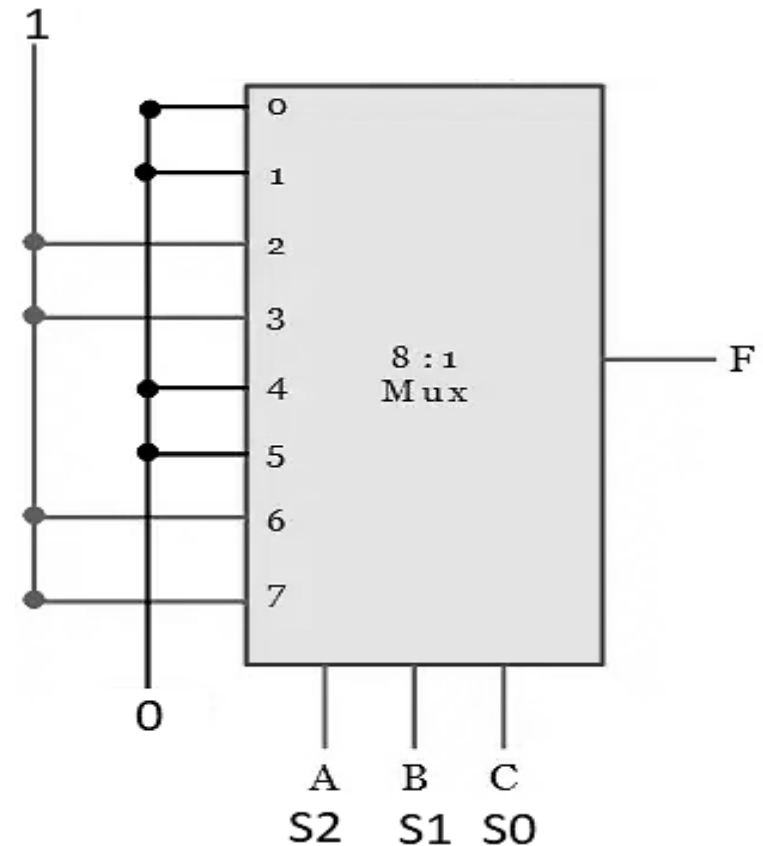
Using an n -input Multiplexer

- For a function with n minterms, use an n -input multiplexer to realize a logic circuit.
- Each minterm of the function can be mapped to an input of the multiplexer.
- For each row in the truth table where the output is 1, set the corresponding input of the multiplexer to 1.
 - That is, for each minterm set the corresponding input of the multiplexer to 1.
- Set the remaining inputs of the multiplexer to 0.

Implement the Boolean expression $F(A, B, C) = \sum m(2, 3, 6, 7)$ using a multiplexer.

$n = 8$; 8:1 mux; 3 Select lines: S2, S1, S0

S2 (A)	S1 (B)	S0 (C)	F
0	0	0	(0) I_0
0	0	1	(0) I_1
0	1	0	(1) I_2
0	1	1	(1) I_3
1	0	0	(0) I_4
1	0	1	(0) I_5
1	1	0	(1) I_6
1	1	1	(1) I_7



Using an $(n/2)$ -input Multiplexer

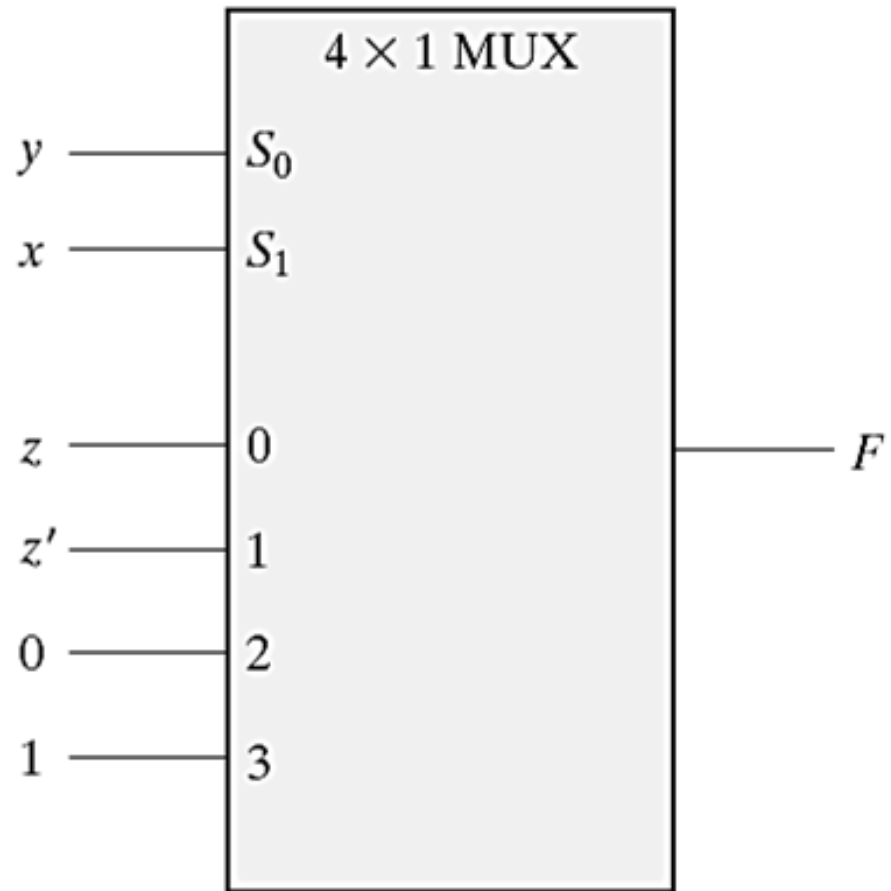
- Use an $(n/2)$ -input multiplexer to realize a logic circuit for a function with n minterms.
- **Group the rows of the truth table, for the function into $(n/2)$ pairs of rows.**
 - Each pair of rows represents a product term
 - Each pair of rows can be mapped to a multiplexer input.
- Determine the logical function of each pair of rows in terms of LSB

Using an $(n / 2)$ -input Mux

Example: $F(x,y,z) = m(1, 2, 6, 7)$

$n = 8$; $n/2 = 4$; 4:1 mux; 2 Select lines: S_1, S_0

S1 S0		i/p	
x	y	z	F
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1



Using an $(n / 2)$ -input Mux

Example: $F(A, B, C) = m(1, 2, 3, 5, 6) + d(7)$

$n = 8$; $n/2 = 4$; 4:1 mux; 2 Select lines: S1, S0

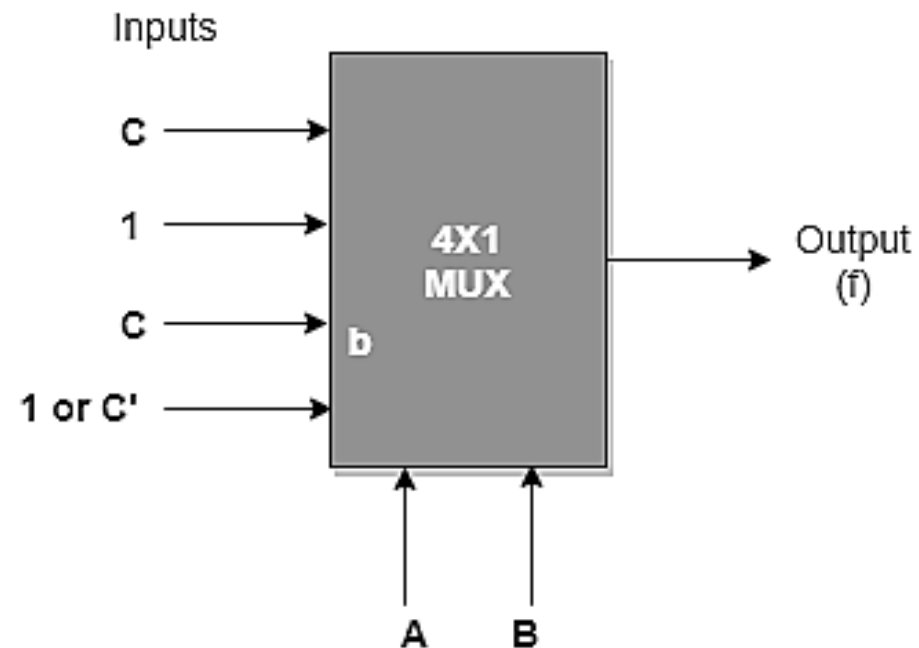
S1 (A)	S0 (B)	i/p (C)	F
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	x

$F = C$

$F = 1$

$F = C$

$F = 1 \text{ or } C'$



Using an $(n / 2)$ -input Mux

Example: $F(A, B, C) = m(1, 2, 3, 5, 6) + d(7)$

$n = 8$; $n/2 = 4$; 4:1 mux; 2 Select lines: S1, S0

AC as select lines

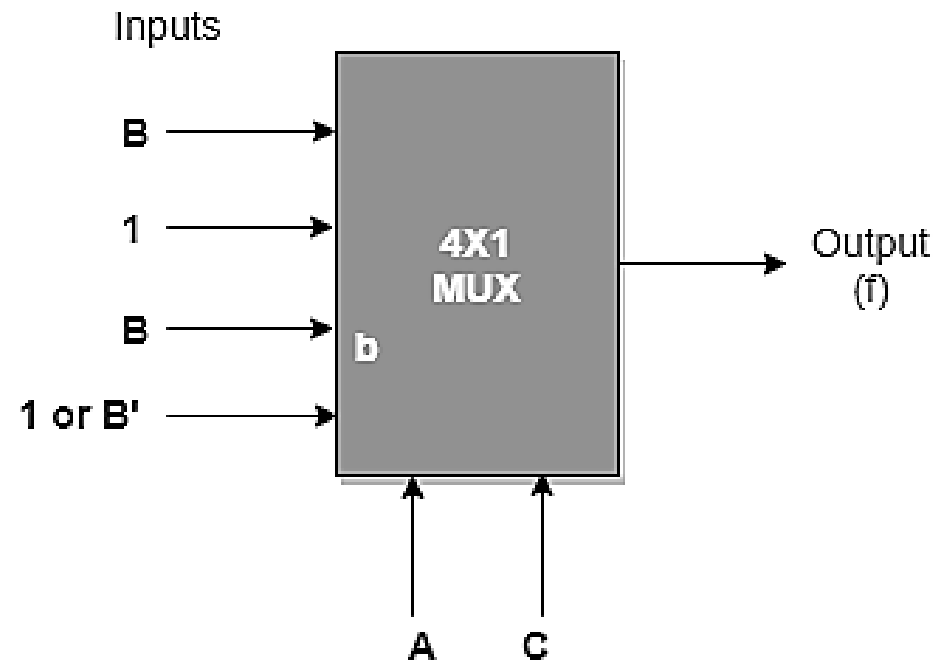
S1 (A)	S0 (C)	i/p (B)	minterm	F
0	0	0	0	0
0	0	1	2	1
0	1	0	1	1
0	1	1	3	1
1	0	0	4	0
1	0	1	6	1
1	1	0	5	1
1	1	1	7	x

$F = B$

$F = 1$

$F = B$

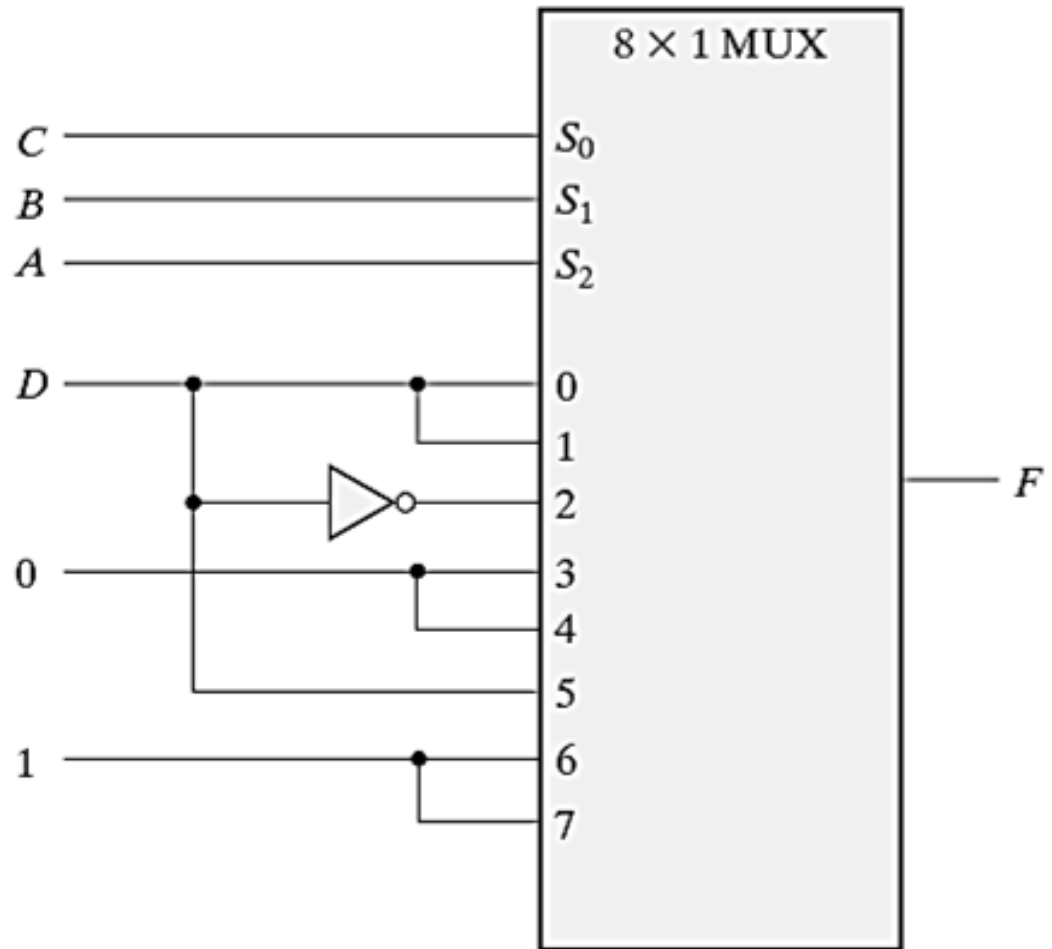
$F = 1 \text{ or } B'$



Using an $(n / 2)$ -input Mux

Example: $F(A,B,C,D) = m(1,3,4,11,12-15)$

A	B	C	D	F	
0	0	0	0	0	$F = D$
0	0	0	1	1	
0	0	1	0	0	$F = D$
0	0	1	1	1	
0	1	0	0	1	$F = D'$
0	1	0	1	0	
0	1	1	0	0	$F = 0$
0	1	1	1	0	
1	0	0	0	0	$F = 0$
1	0	0	1	0	
1	0	1	0	0	$F = D$
1	0	1	1	1	
1	1	0	0	1	$F = 1$
1	1	0	1	1	
1	1	1	0	1	$F = 1$
1	1	1	1	1	



Using an (n / 4)-input Mux

Example: $F(A, B, C) = m(1, 2, 3, 5, 6) + d(7)$

$n = 8$; $n/4 = 2$; 2:1 mux; 1 Select line: S_0 ;

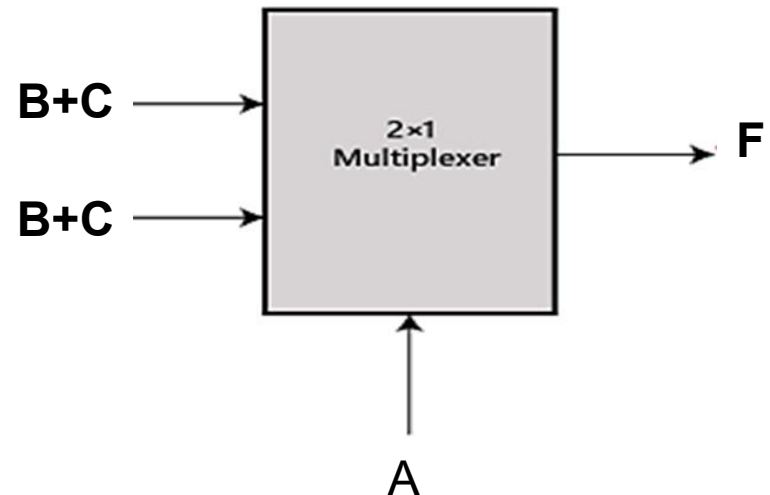
A : select line,

B, C : i/p

S_0 (A)	i/p (B)	i/p (C)	F
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	x

$$F = B + C$$

$$F = B + C \text{ or } B'C + BC'$$



Gate Level Modeling : 2:1 MUX

```
module multiplexer (A, B, Sel, Out);
```

```
input A, B, Sel;
```

```
output Out;
```

```
wire Selbar, tmp1, tmp0;
```

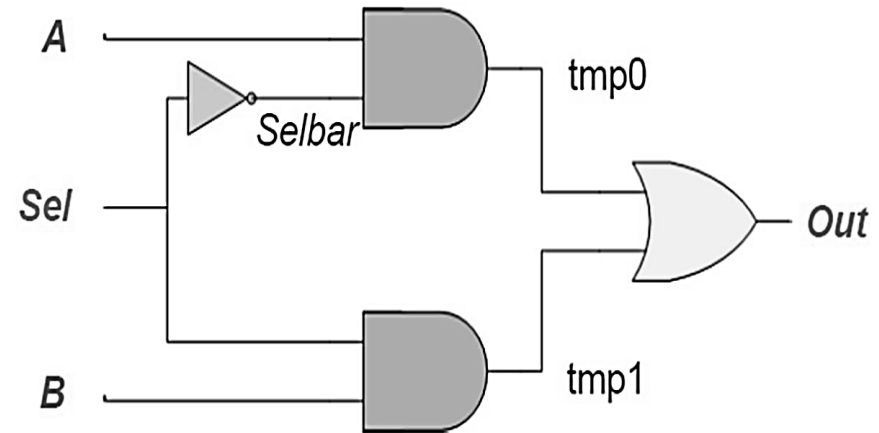
```
and a0 (tmp1, B, Sel);
```

```
not (Selbar, Sel);
```

```
and a1 (tmp0, A, Selbar);
```

```
or (Out, tmp0, tmp1);
```

```
endmodule
```



Gate Level Modeling: 4-to-1 Multiplexer

//4-to-1 Mux: Structural Verilog

```
module mux_4_to_1_st_v(S,D,Y);
```

```
  input [1:0]S;
```

```
  input [3:0]D;
```

```
  output Y;
```

```
  wire [1:0]not_s;
```

```
  wire [0:3]w;
```

```
  not g0(not_s[0],S[0]),
```

```
    g1(not_s[1],S[1]);
```

```
  and g2(w[0], not_s[1], not_s[0], D[0]),
```

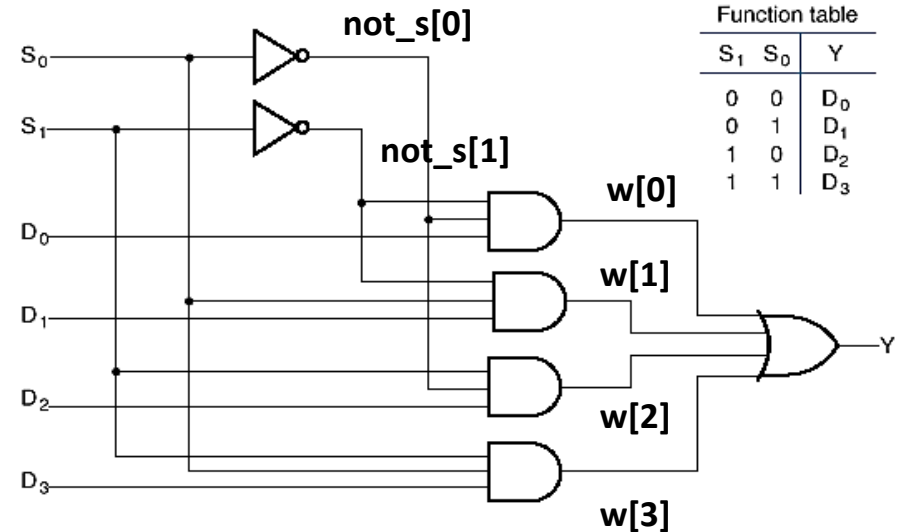
```
    g3(w[1], not_s[1], S[0], D[1]),
```

```
    g4(w[2], S[1], not_s[0], D[2]),
```

```
    g5(w[3], S[1],S[0],D[3]);
```

```
  or g6(Y,w[0],w[1],w[2],w[3]);
```

```
endmodule
```



Dataflow Modeling – 2:1 Multiplexer

$$Out = \overline{Sel} \cdot A + Sel \cdot B$$

// function modeled as a logic expression

```
module MUX2 (A,B,S,Z);
```

```
input A,B,S;                                //input ports
```

```
output Z;                                    //output port
```

```
    assign Z = (~S & A) | (S & B); //continuous evaluation
```

```
endmodule
```

Using conditional operator:

```
assign Z = (S == 0) ? A : B;
```

True/false
condition

if true : if false

Dataflow Modeling – 4:1 Multiplexer

$$Y = S_1'S_0'D_0 + S_1'S_0D_1 + S_1S_0'D_2 + S_1S_0D_3$$

//4-to-1 Mux: Boolean Expression

```
module mux_4_to_1(S,D,Y);  
input [1:0]S;  
input [3:0]D;  
output Y;  
assign Y = (~S[1]&~S[0]&D[0]) | (~S[1]&S[0]&D[1])  
           | (S[1]&~S[0]&D[2]) | (S[1]&S[0]&D[3]);  
endmodule
```

Dataflow Modeling – 4:1 Multiplexer

//4-to-1 Mux: Dataflow Verilog Description, conditional

```
module mux_4_to_1(S,D,Y);
```

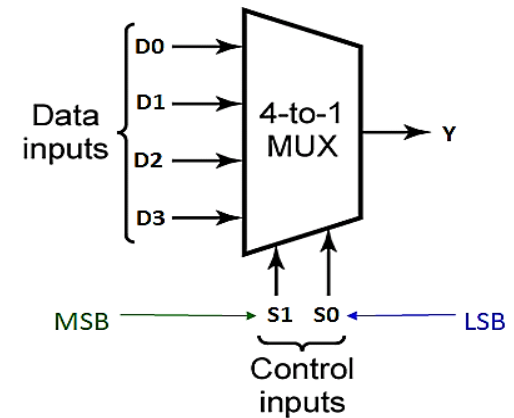
```
    input [1:0]S;
```

```
    input [3:0]D;
```

```
    output Y;
```

```
    assign Y=S[1] ? (S[0]?D[3]:D[2]) : (S[0]?D[1]:D[0]);
```

```
endmodule
```



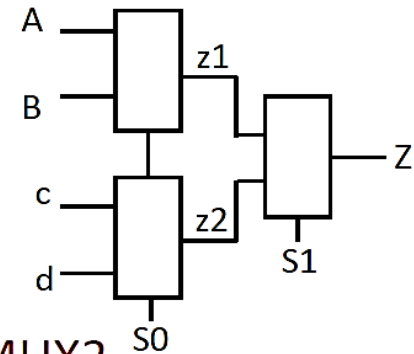
Function table		
S ₁	S ₀	Y
0	0	D ₀
0	1	D ₁
1	0	D ₂
1	1	D ₃

4:1 Multiplexer using 2:1 Mux

Define MUX2 module in Verilog source before compiling MUX4 module

```
module MUX2 (A,B,S,Z);  
input A,B,S;  
output Z;  
    assign Z = (~S & A) | (S & B);  
endmodule
```

```
module MUX4 (A,B,c,d,S0,S1,Z);  
    input A,B,c,d,S0,S1;  
    output Z;  
    wire z1,z2;  
    MUX2 M1(A,B,S0,z1);  
    MUX2 M2(c,d,S0,z2);  
    MUX2 M3(z1,z2,S1,Z)  
endmodule
```



Test-Bench Mux 2:1

//Stimulus for mux2x1_df

module testmux;

reg TA,TB,TS; //inputs for mux

wire Y; //output from mux

mux2x1_df mx (TA,TB,TS,Y); // instantiate mux

initial

begin

\$monitor("select=%b A=%b B=%b OUT=%b",TS,TA,TB,Y);

TS = 1; TA = 0; TB = 1;

#100 TA = 1; TB = 0;

#100 TS = 0;

#100 TA = 0; TB = 1;

end

endmodule

Test-Bench Mux 4:1

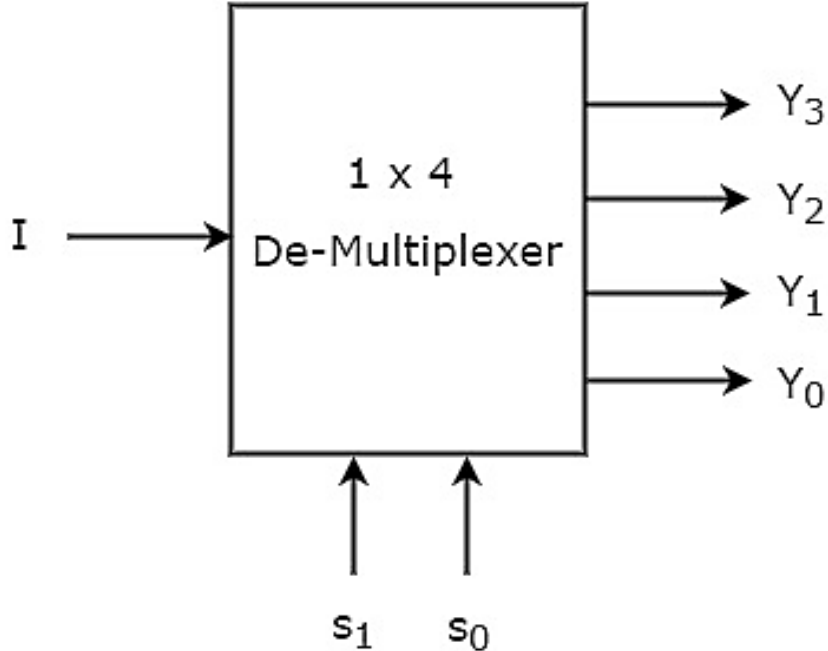
```
module tb;
  reg [1:0] sel;
  reg i0,i1,i2,i3;
  wire y;
  mux_example mux(sel, i0, i1, i2, i3, y);
  initial
  begin
    $monitor("sel = %b -> i3 = %0b, i2 = %0b ,i1 = %0b, i0 = %0b -> y = %0b",
sel,i3,i2,i1,i0, y);
    {i3,i2,i1,i0} = 4'h5;
    repeat(6) begin
      sel = $random;
      #100;
    end
  end
endmodule
```

Demultiplexer (DEMUX)

Demultiplexers

- A demultiplexer has
 - N control inputs
 - 1 data input
 - 2^N outputs
- A demultiplexer routes (or connects) the data input to the selected output.
 - The value of the control inputs determines the output that is selected.
- A demultiplexer performs the opposite function of a multiplexer.

1:4 Demultiplexer



SELECT		OUTPUTS			
s_1	s_0	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	I
0	1	0	0	I	0
1	0	0	I	0	0
1	1	I	0	0	0

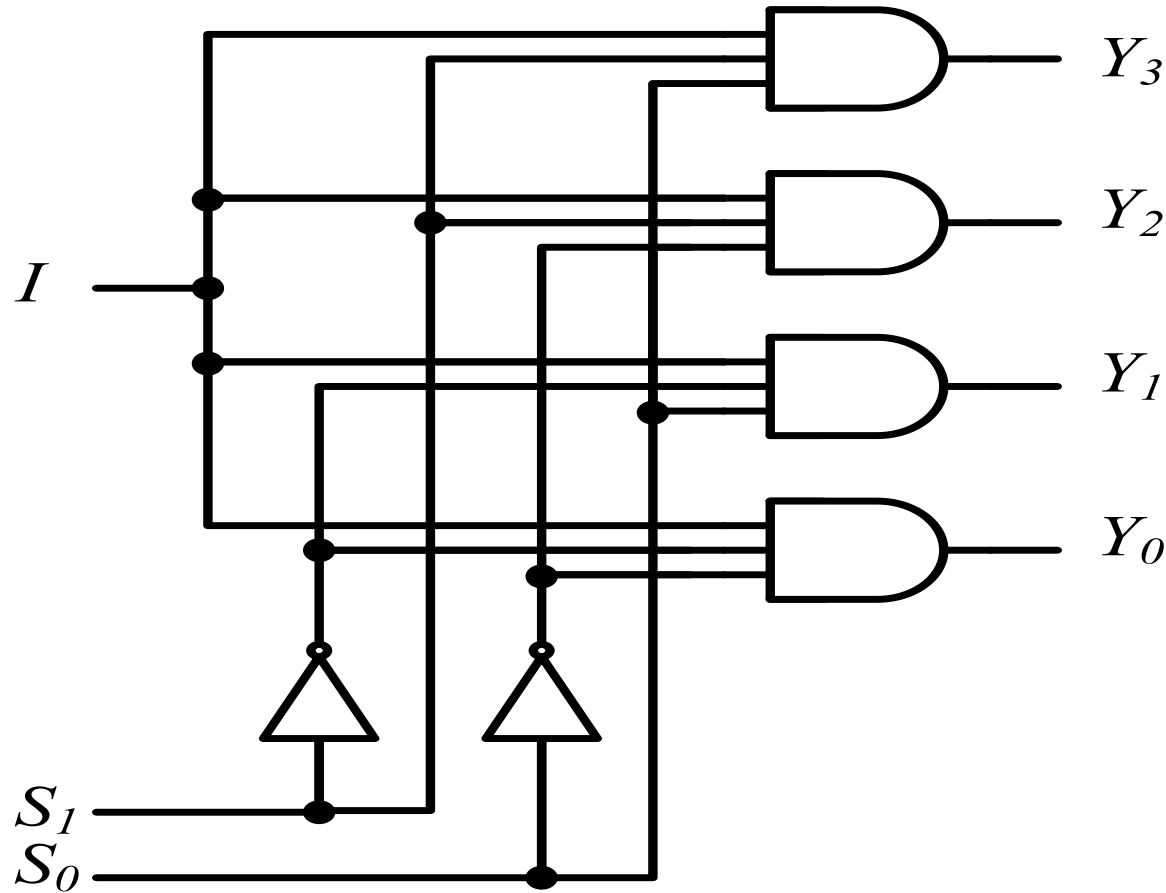
$$Y_3 = s_1 s_0 I$$

$$Y_2 = s_1 s_0' I$$

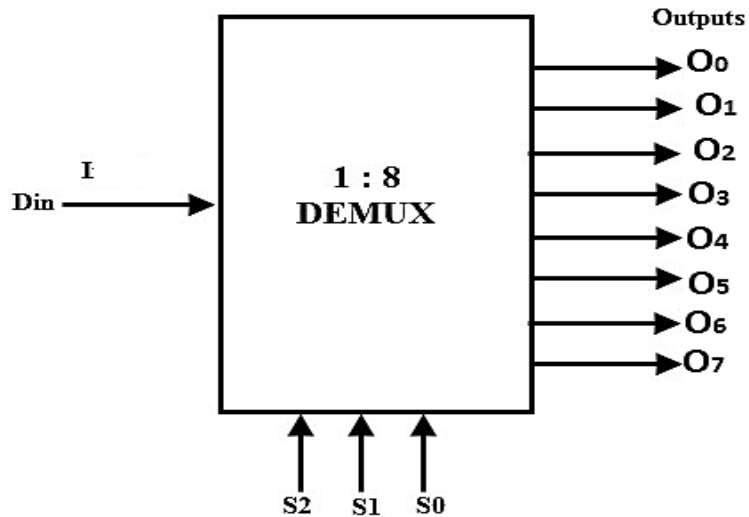
$$Y_1 = s_1' s_0 I$$

$$Y_0 = s_1' s_0' I$$

Demultiplexer 1:4 - Combinational circuit

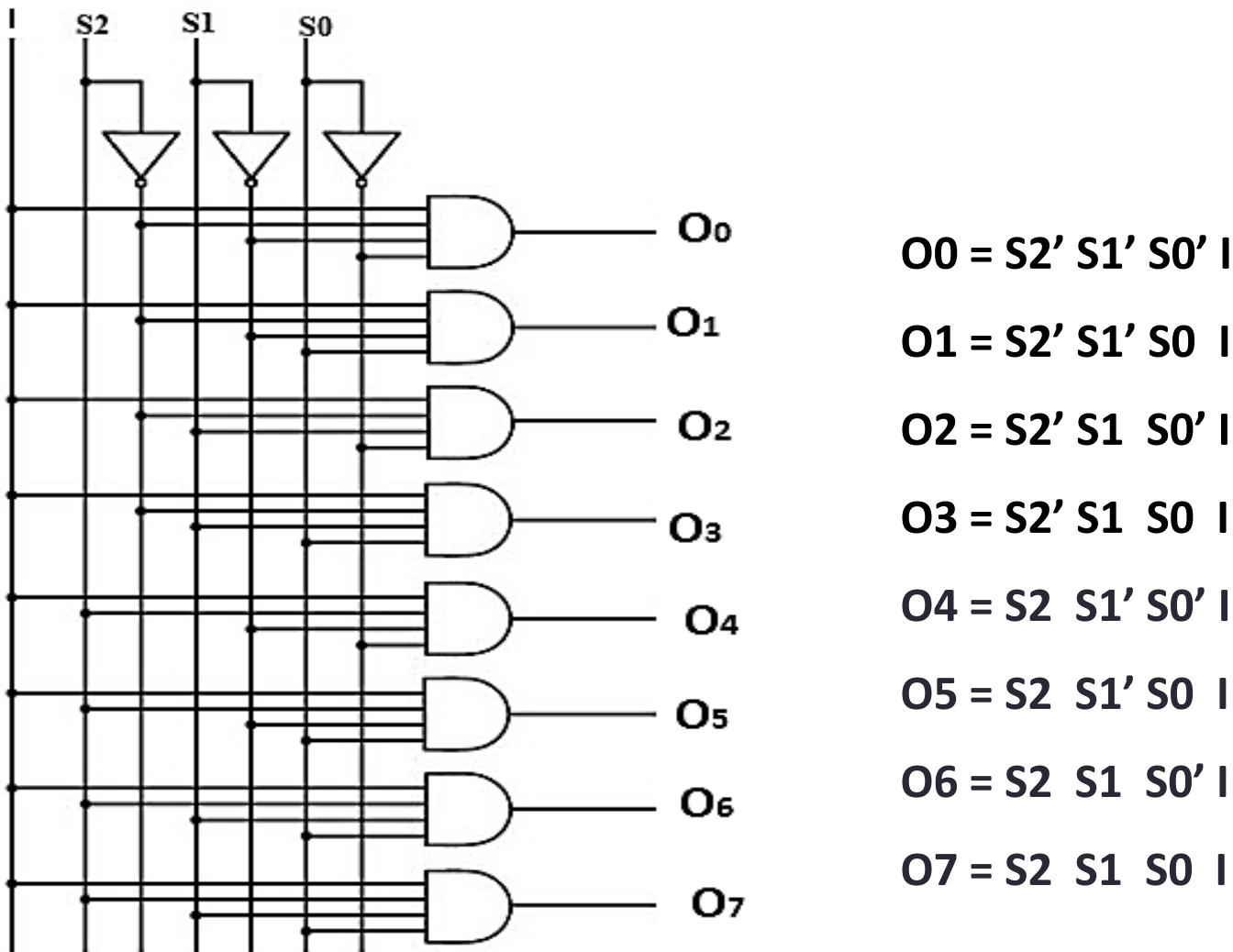


1:8 Demultiplexers

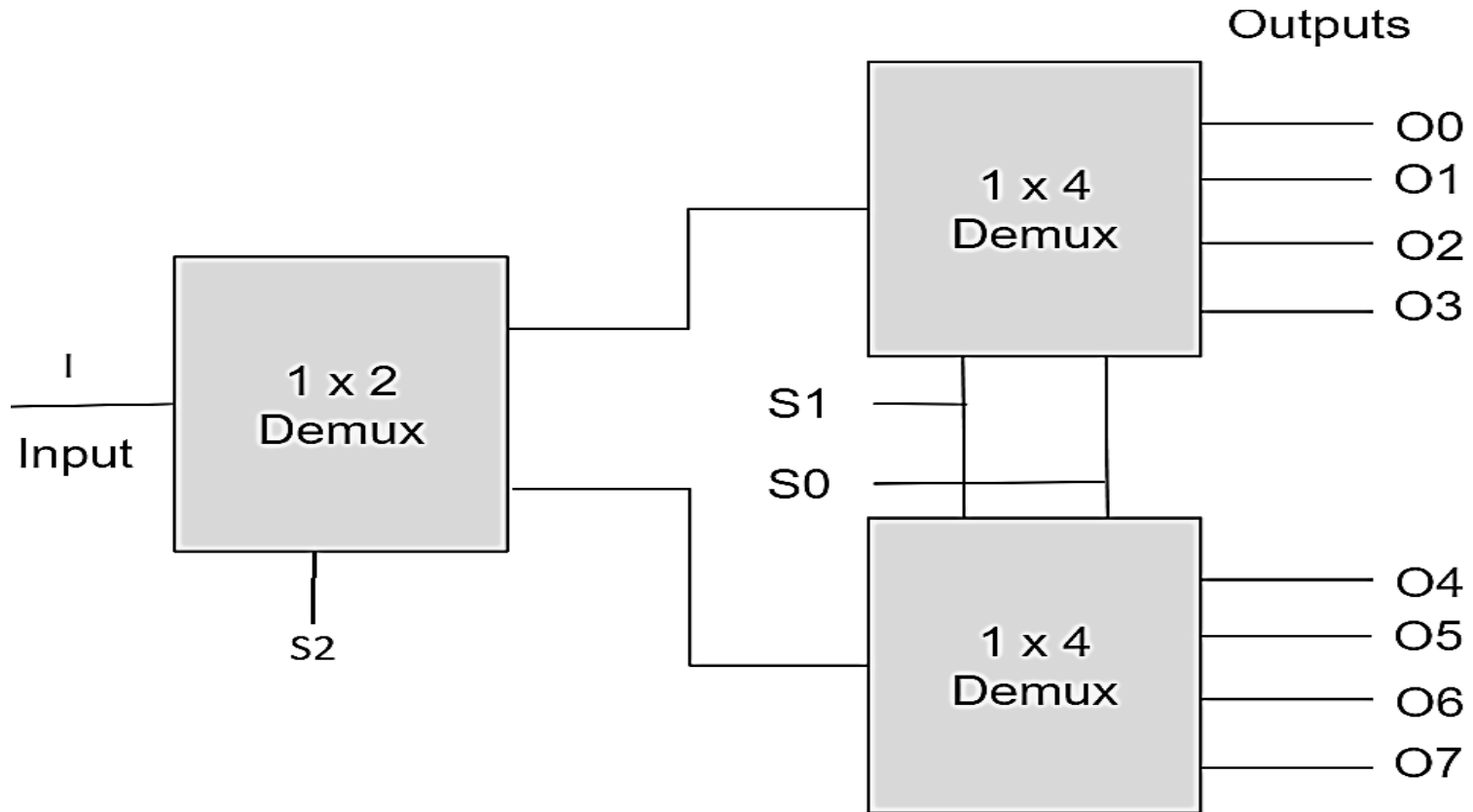


SELECT			OUTPUTS							
S2	S1	S0	O7	O6	O5	O4	O3	O2	O1	O0
0	0	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	1	0
0	1	0	0	0	0	0	0	1	0	0
0	1	1	0	0	0	0	1	0	0	0
1	0	0	0	0	0	1	0	0	0	0
1	0	1	0	0	1	0	0	0	0	0
1	1	0	0	1	0	0	0	0	0	0
1	1	1	1	0	0	0	0	0	0	0

1:8 Demux Logic diagram



1:8 DeMux using 1:4, 1:2 Demux

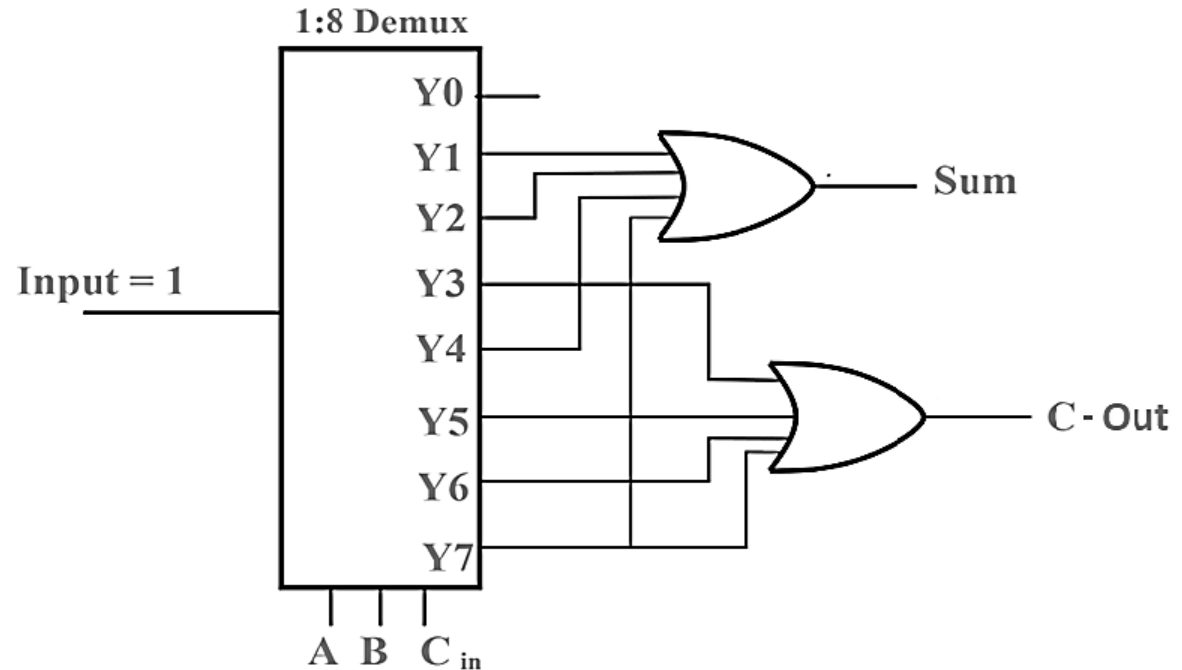


Design Full adder using 1:8 Demux

Inputs			Outputs	
A	B	C-IN	Sum	C-Out
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

$$S(A,B,C_{in}) = \sum(1,2,4,7)$$

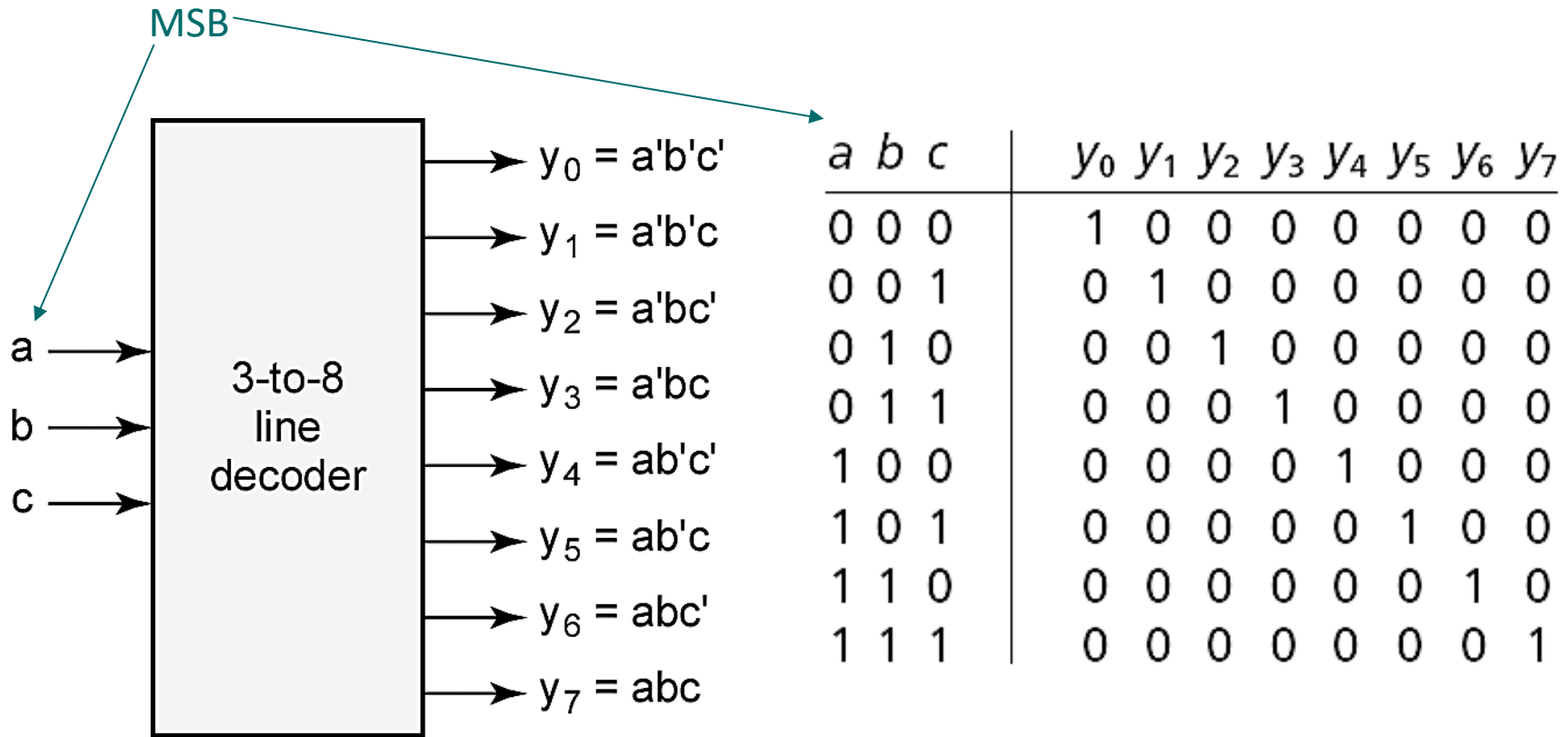
$$C_{out}(A,B,C_{in}) = \sum(3,5,6,7)$$



Decoders

- A decoder has
 - N inputs
 - 2^N outputs
- Ex: 2:4, 3:8 Decoder
- A decoder selects one of 2^N outputs by decoding the binary value on the N inputs.
- The decoder generates all of the minterms of the N input variables.
 - Exactly one output will be active for each combination of the inputs.

3:8 Decoder without Enable

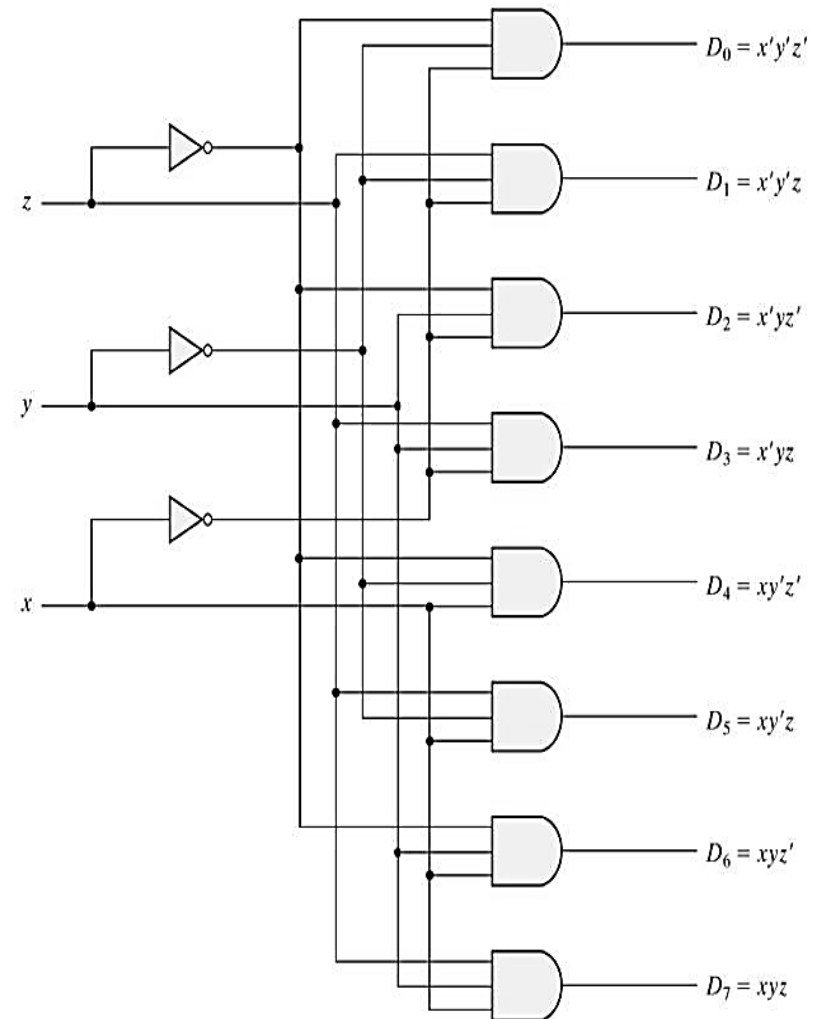


- The three inputs are decoded into eight outputs, each representing one of the minterms of the three input variables.
- For each possible input combination, there are seven outputs that are equal to 0 and **only one that is equal to 1** representing the minterm

3:8 Decoder without Enable

- Combinational circuit that converts $N = 3$ lines of input into $2^n = 8$ lines of output

Inputs			Outputs							
x	y	z	D0	D1	D2	D3	D4	D5	D6	D7
0	0	0	1	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0	0	0	0
0	1	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	1	0	0	0	0
1	0	0	0	0	0	0	1	0	0	0
1	0	1	0	0	0	0	0	1	0	0
1	1	0	0	0	0	0	0	0	1	0
1	1	1	0	0	0	0	0	0	0	1



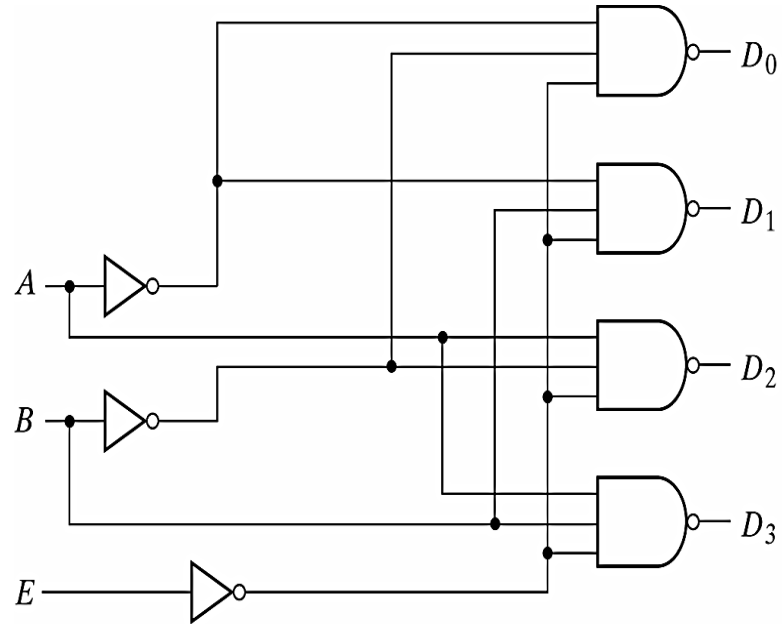
- Each one of the eight **AND gates** generates one of the minterms.
- A particular application of this decoder is **binary-to-octal** conversion.

Decoder with Enable

- Some decoders are constructed with NAND gates.
- It becomes more economical to generate the decoder minterms in their complemented form.
- Furthermore, decoders include one or more enable inputs to control the circuit operation.
- The circuit operates with complemented outputs and a complement enable input.
- The outputs of the decoder are enabled when **E is equal to 0 (i.e., active-low enable)**.
- The outputs of the decoder are enabled when **E is equal to 1 (i.e., active-high enable)**.

2 to 4 Decoder with Enable

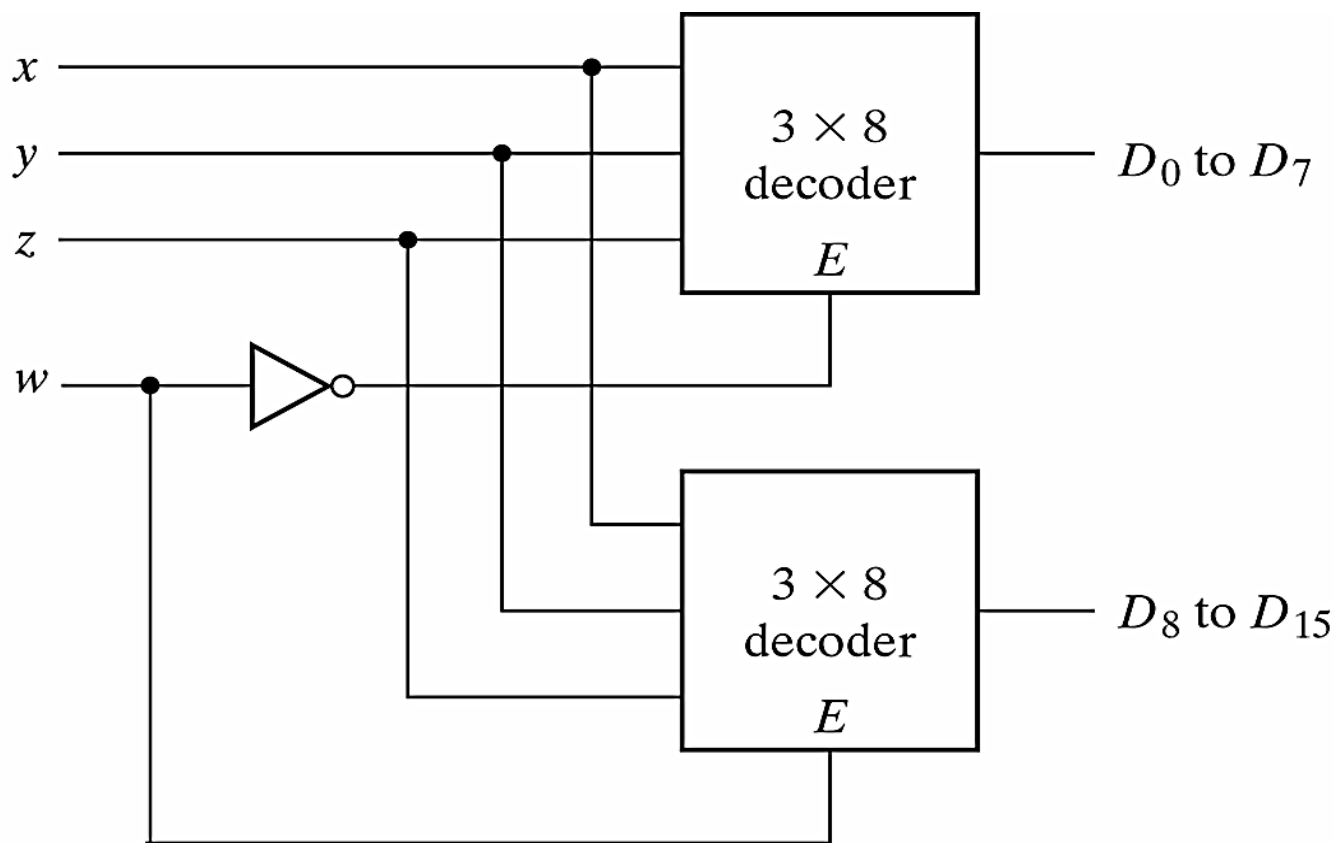
E	A	B	D_0	D_1	D_2	D_3
1	X	X	1	1	1	1
0	0	0	0	1	1	1
0	0	1	1	0	1	1
0	1	0	1	1	0	1
0	1	1	1	1	1	0



- Only one output can be equal to 0 at any given time; all other outputs are equal to 1.
- The output whose value is equal to 0 represents the minterm selected by inputs A and B.
- The circuit is disabled when E is equal to 1, regardless of the values of the other two inputs.
- When the circuit is disabled, none of the outputs are equal to 0 and none of the minterms are selected.

4-to-16-line Decoder constructed with two 3-to-8-line decoders

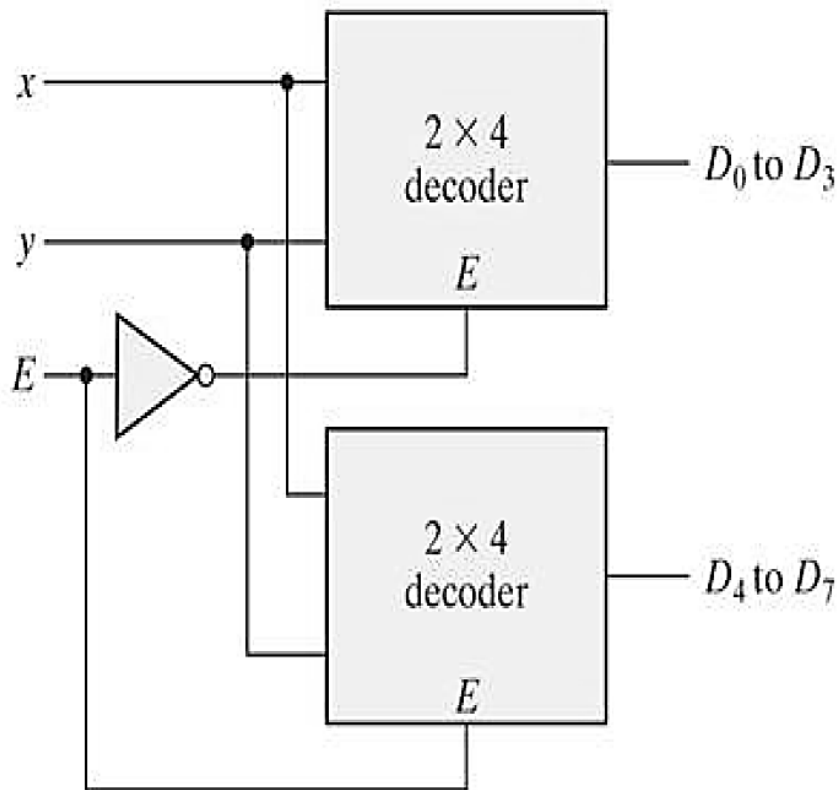
Decoders with enable inputs can be connected together to form a larger decoder.



4-line-to-16-line Decoder constructed with two 3-line-to-8-line decoders (cont.)

- When $w=0$, the top decoder is enabled and the other is disabled. The bottom decoder outputs are all 0's, and the top eight outputs generate min-terms 0000 to 0111.
- When $w=1$, the enable conditions are reversed. The bottom decoder outputs generate min-terms 1000 to 1111, while the outputs of the top decoder are all 0's.

Draw a logic diagram constructing a 3×8 decoder with active-low enable, using a pair of 2×4 decoders; also draw a truth table for the configuration.



E	x	y	D_0	D_1	D_2	D_3	D_4	D_5	D_6	D_7
0	0	0	1	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0	0	0	0
0	1	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	1	0	0	0	0
1	0	0	0	0	0	0	1	0	0	0
1	0	1	0	0	0	0	0	1	0	0
1	1	0	0	0	0	0	0	0	1	0
1	1	1	0	0	0	0	0	0	0	1

Boolean Function Implementation / Design of logic circuits using Decoders

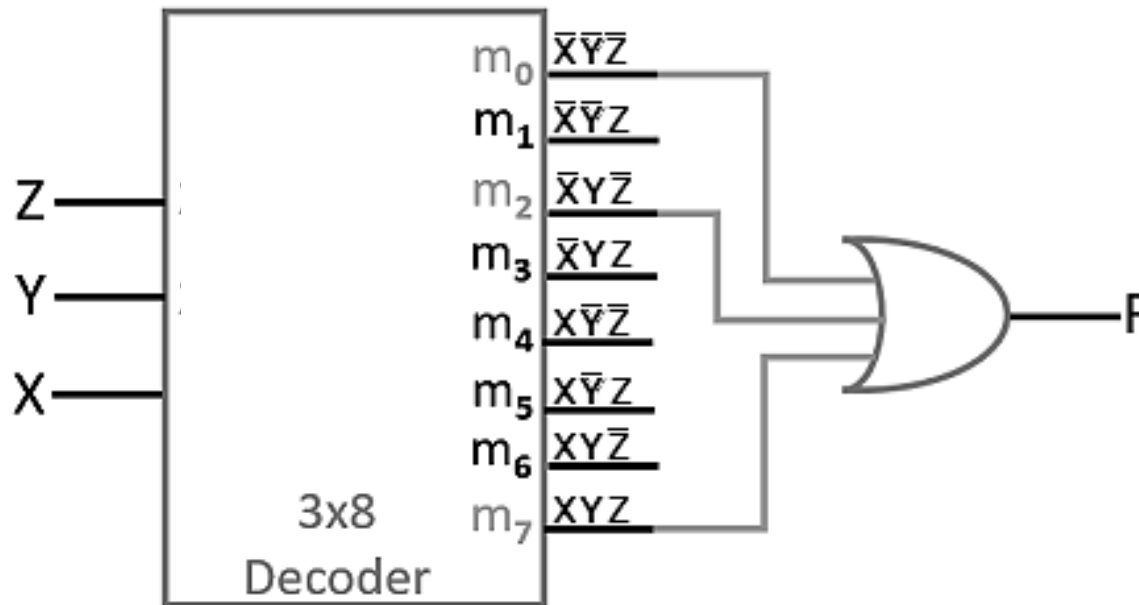
Using an n -output Decoder

- Use an n -output decoder to realize a logic circuit for a function with n minterms.
- **Each minterm of the function can be mapped to an output of the decoder.**
- For each row in the truth table where the output is 1, sum (“OR”) the corresponding outputs of the decoder.
 - That is, for each minterm in the minterm expansion of the function, OR the corresponding outputs of the decoder.
- Leave remaining outputs of the decoder unconnected.

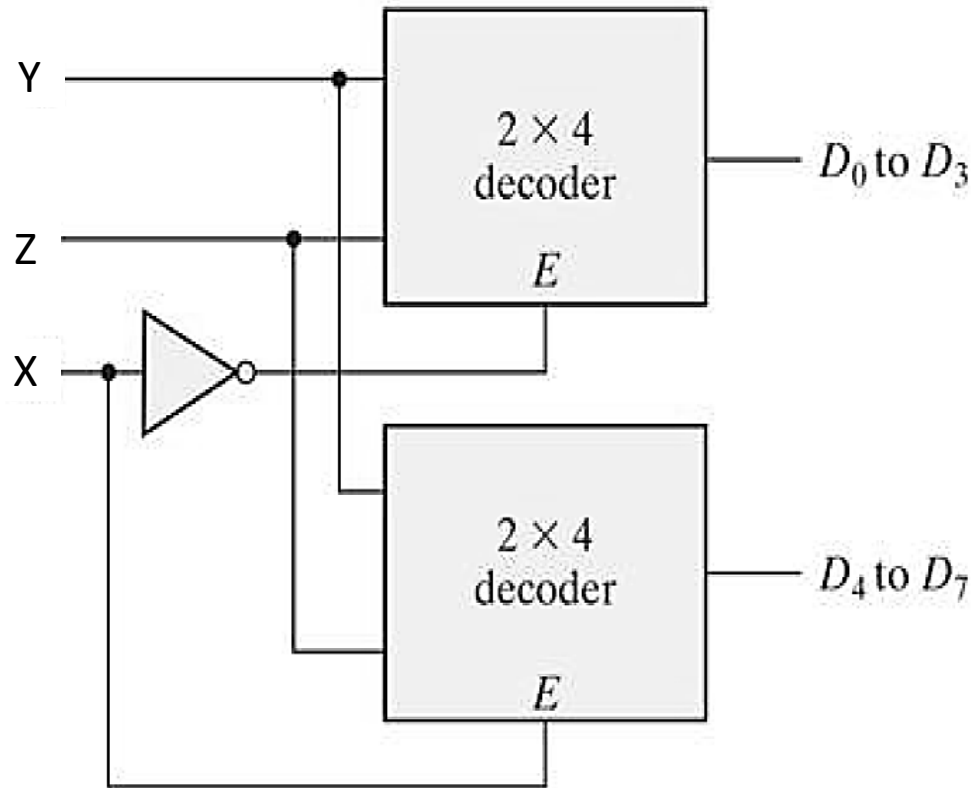
Using an n -output Decoder

Example: Using a 3-to-8 decoder, design a logic circuit to realize the following Boolean function

$$F(X,Y,Z) = m(0, 2, 7)$$



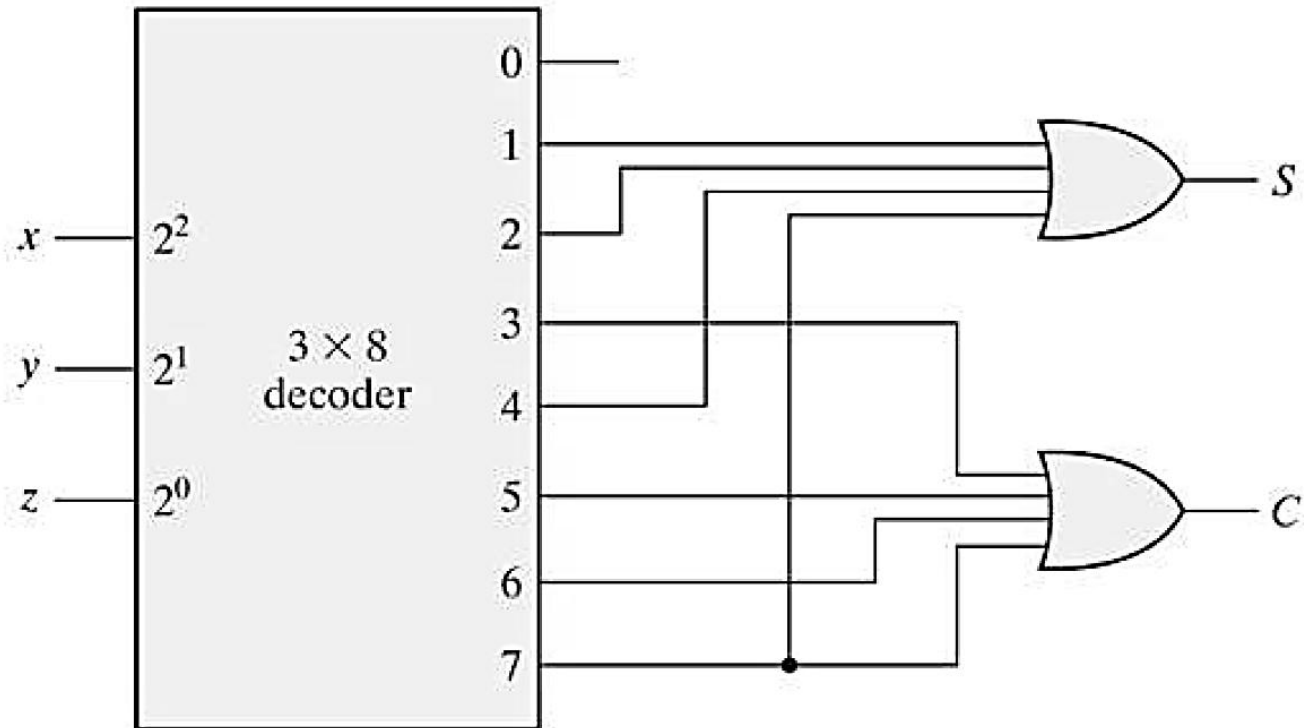
Using $n/2$ -output Decoder



Design a full adder using decoder

Truth Table of Full Adder

Inputs			Outputs	
X	Y	Z	C	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1



$$S(x, y, z) = \Sigma(1, 2, 4, 7)$$

$$C(x, y, z) = \Sigma(3, 5, 6, 7)$$

Gate Level Modeling: 2 to 4 Decoder

//Gate-level description of a 2-to-4-line decoder

```
module decoder_gl (A,B,E,D);
```

```
  input  A,B,E;
```

```
  output [0:3]D;
```

```
  wire  Anot,Bnot,Enot;
```

```
  not n1 (Anot,A),
```

```
      n2 (Bnot,B),
```

```
      n3 (Enot,E);
```

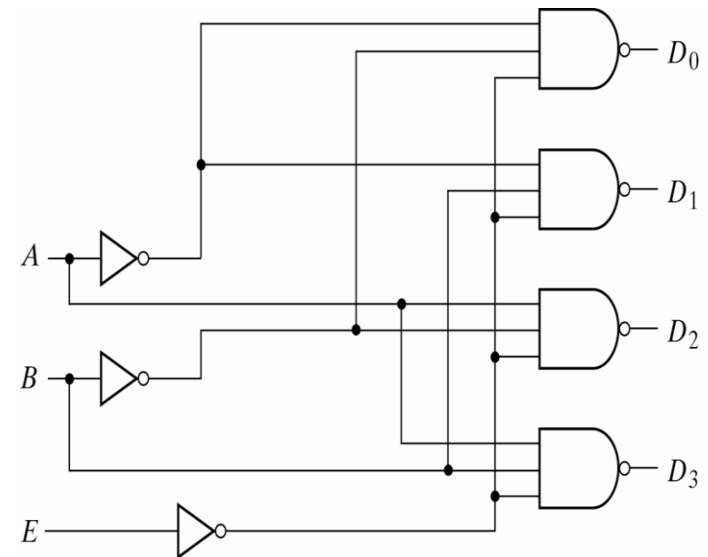
```
  nand n4 (D[0],Anot,Bnot,Enot),
```

```
      n5 (D[1],Anot,B,Enot),
```

```
      n6 (D[2],A,Bnot,Enot),
```

```
      n7 (D[3],A,B,Enot);
```

```
endmodule
```



Dataflow Modeling: 2 to 4 Decoder

//Dataflow description of a 2-to-4-line decoder

```
module decoder_df (A,B,E,D);
```

```
  input A,B,E;
```

```
  output [0:3] D;
```

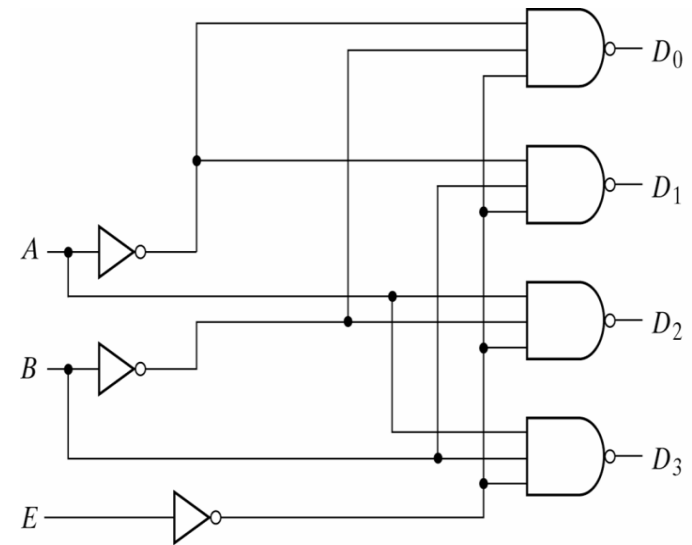
```
  assign D[0] = ~(~A & ~B & ~E),
```

```
          D[1] = ~(~A & B & ~E),
```

```
          D[2] = ~(A & ~B & ~E),
```

```
          D[3] = ~(A & B & ~E);
```

```
endmodule
```



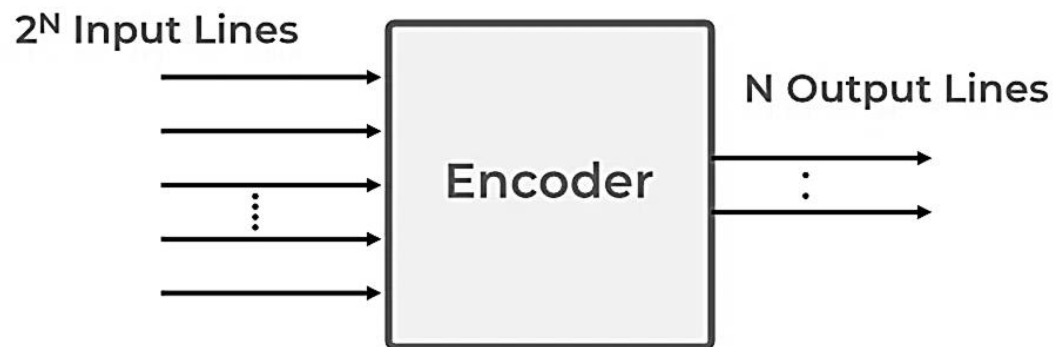
Test-Bench 3:8 Decoder

```
module tb;
  reg [2:0] D;
  wire [7:0] y;
  binary_decoder bin_dec(D, y);
  initial
  begin
    $monitor("D = %b -> y = %0b", D, y);
    repeat(5) begin
      D=$random;
      #100;
    end
  end
endmodule
```

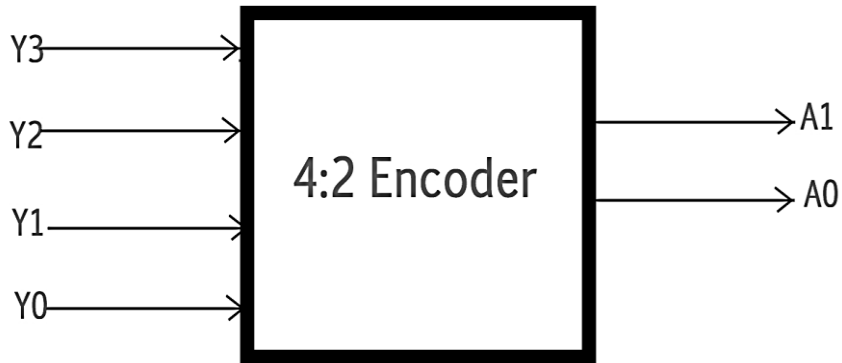
Encoders

Encoders

- An encoder has
 - 2^N inputs
 - N outputs
- An encoder outputs the binary value of the selected (or active) input.
- An encoder performs the inverse operation of a decoder.



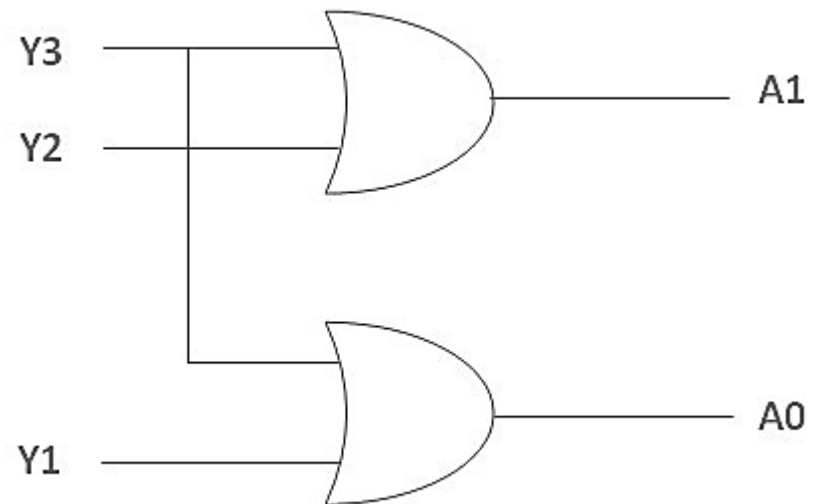
4 to 2 Encoder



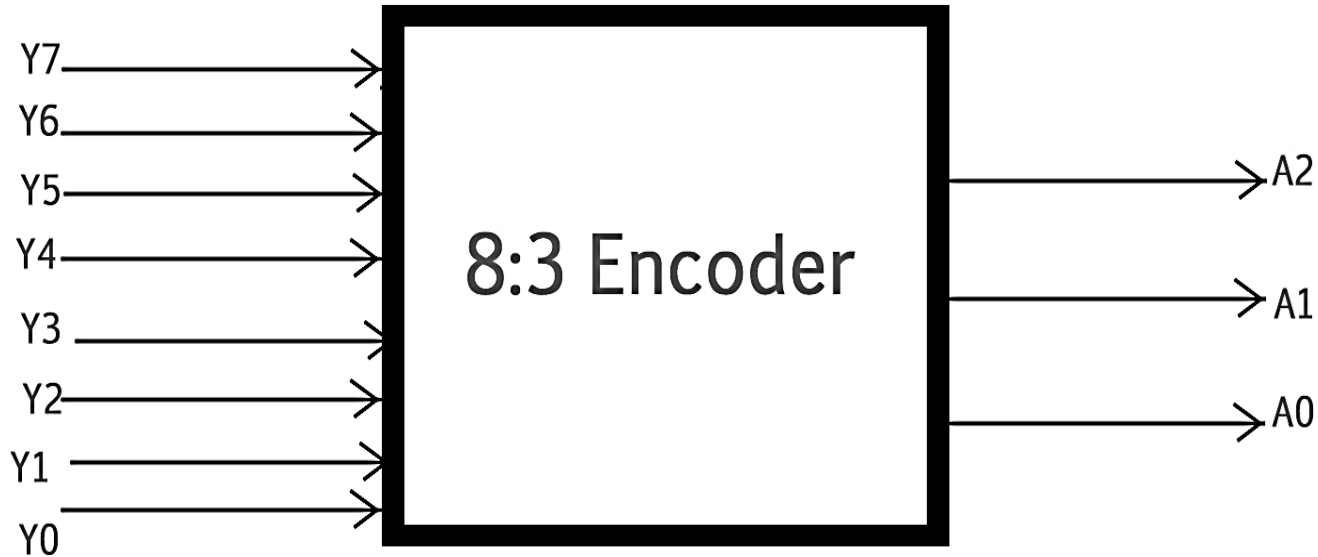
INPUTS				OUTPUTS	
Y3	Y2	Y1	Y0	A1	A0
0	0	0	1	0	0
0	0	1	0	0	1
0	1	0	0	1	0
1	0	0	0	1	1

$$A1 = Y3 + Y2$$

$$A0 = Y3 + Y1$$



8: 3 (Octal to Binary) Encoder



- An octal to binary encoder has 8 inputs and 3 outputs (assume only one input is high (1) at a given time).
- Limitation : If multiple inputs are high output produces an undefined combination.

8: 3 (Octal to Binary) Encoder

INPUTS								OUTPUTS		
Y7	Y6	Y5	Y4	Y3	Y2	Y1	Y0	A2	A1	A0
0	0	0	0	0	0	0	1	0	0	0
0	0	0	0	0	0	1	0	0	0	1
0	0	0	0	0	1	0	0	0	1	0
0	0	0	0	1	0	0	0	0	1	1
0	0	0	1	0	0	0	0	1	0	0
0	0	1	0	0	0	0	0	1	0	1
0	1	0	0	0	0	0	0	1	1	0
1	0	0	0	0	0	0	0	1	1	1

$$A2 = Y7 + Y6 + Y5 + Y4$$

$$A1 = Y7 + Y6 + Y3 + Y2$$

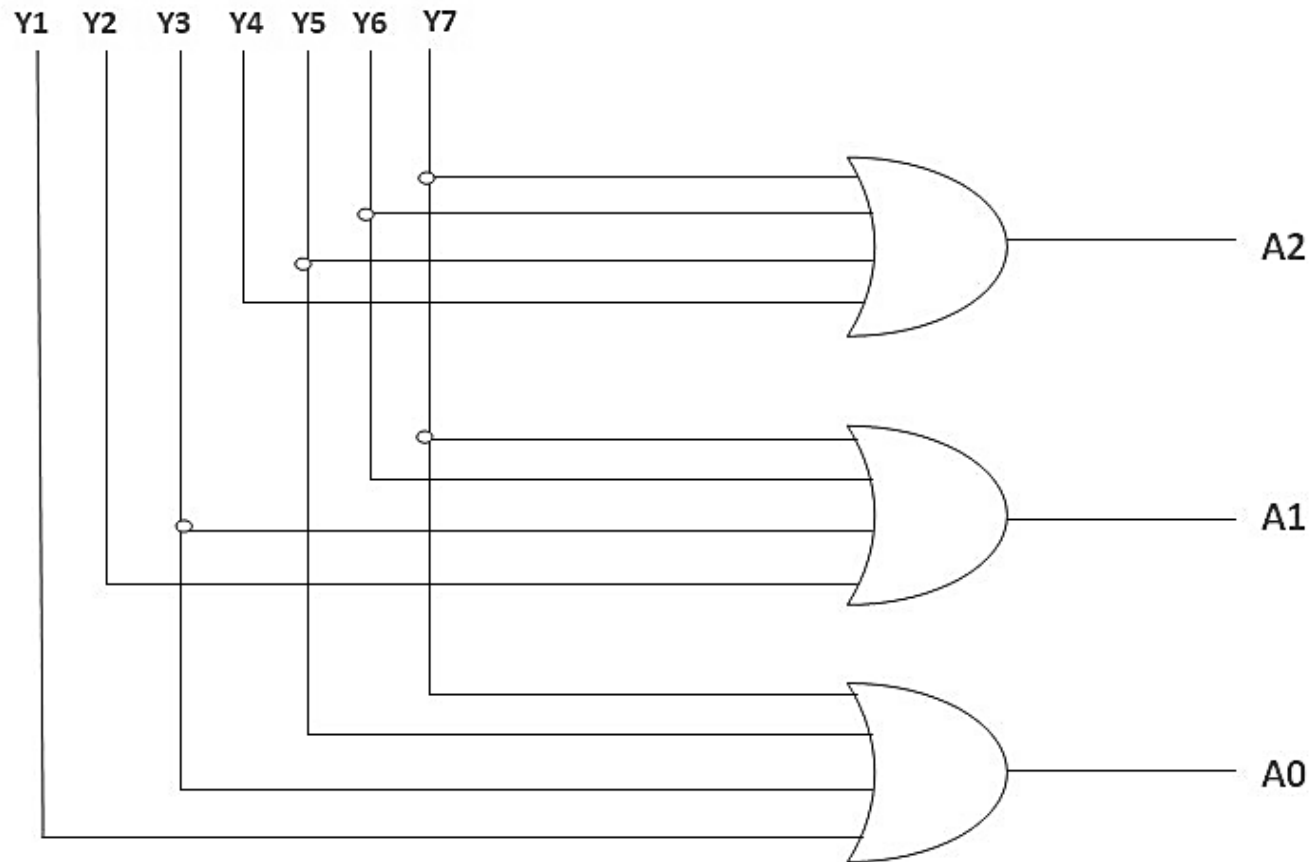
$$A0 = Y7 + Y5 + Y3 + Y1$$

$$A2 = Y7 + Y6 + Y5 + Y4$$

$$A1 = Y7 + Y6 + Y3 + Y2$$

$$A0 = Y7 + Y5 + Y3 + Y1$$

An octal to binary encoder (8:3 encoder) can be implemented using three OR gates.



8:3 Binary Encoder Verilog Code

```
module binary_encoder(  
    input [7:0] D,  
    output [2:0] y);  
  
    assign y[2] = D[4] | D[5] | D[6] | D[7];  
    assign y[1] = D[2] | D[3] | D[6] | D[7];  
    assign y[0] = D[1] | D[3] | D[5] | D[7];  
  
endmodule
```

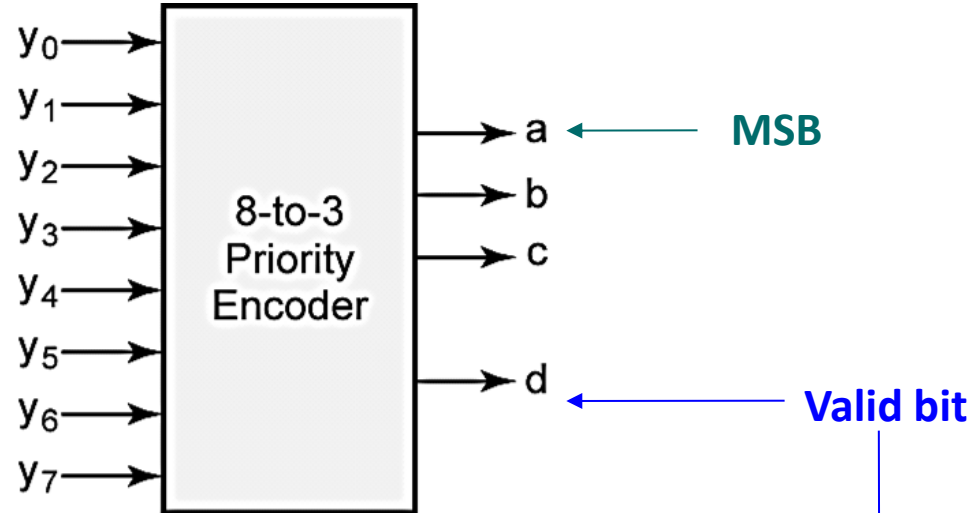
Testbench Code - 8:3 Encoder

```
module encoder_8_3_tb;
  reg [7:0] D;
  wire [2:0] y;
  integer i;
  binary_encoder bin_enc(D, y);
  initial begin
    D=8'b1; #100;
    for(i=0; i<8; i=i+1)
begin
    $display("D = %h(in dec:%0d) -> y = %0h", D, i, y);
    D=D<<1; #100;
  end
end
endmodule
```

Priority Encoders

- If more than one input is active, the higher-order input has priority over the lower-order input.
 - The higher value is encoded on the output
- A valid indicator, d , is included to indicate whether or not the output is valid.
 - Output is invalid when no inputs are active
 - $d = 0$
 - Output is valid when at least one input is active
 - $d = 1$

Priority Encoders



y_7	y_6	y_5	y_4	y_3	y_2	y_1	y_0	a	b	c	d
0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	1	0	0	0	1
0	0	0	0	0	0	1	X	0	0	1	1
0	0	0	0	0	1	X	X	0	1	0	1
0	0	0	0	1	X	X	X	0	1	1	1
0	0	0	1	X	X	X	X	1	0	0	1
0	0	1	X	X	X	X	X	1	0	1	1
0	1	X	X	X	X	X	X	1	1	0	1
1	X	X	X	X	X	X	X	1	1	1	1

4:2 Priority Encoder – Truth Table and K- Map

INPUTS				OUTPUTS		
Y3	Y2	Y1	Y0	A1	A0	V
0	0	0	0	X	X	0
0	0	0	1	0	0	1
0	0	1	X	0	1	1
0	1	X	X	1	0	1
1	X	X	X	1	1	1

4:2 Priority Encoder – Truth Table and K- Map

INPUTS				OUTPUTS		
Y3	Y2	Y1	Y0	A1	A0	V
0	0	0	0	X	X	0
0	0	0	1	0	0	1
0	0	1	X	0	1	1
0	1	X	X	1	0	1
1	X	X	X	1	1	1

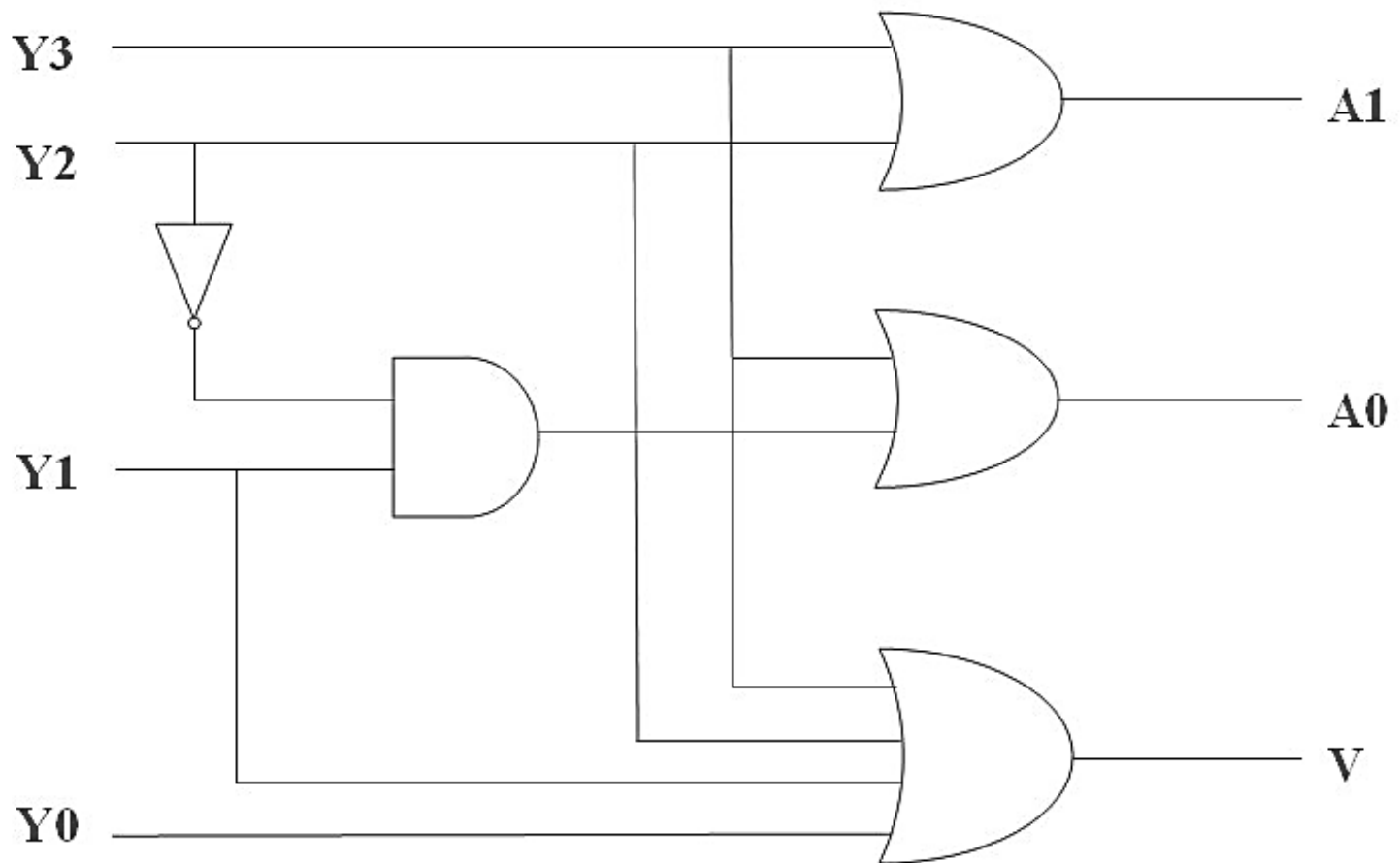
Y3	Y2	Y1	Y0	A1	A0	V
0	0	0	0	X	X	0
0	0	0	1	0	0	1
0	0	1	0	0	1	1
0	0	1	1	0	1	1
0	1	0	0	1	0	1
0	1	0	1	1	0	1
0	1	1	0	1	0	1
0	1	1	1	1	0	1
1	0	0	0	1	1	1
1	0	0	1	1	1	1
1	0	1	0	1	1	1
1	0	1	1	1	1	1
1	1	0	0	1	1	1
1	1	0	1	1	1	1
1	1	1	0	1	1	1
1	1	1	1	1	1	1

		Y1 Y0			
		00	01	11	10
Y3 Y2	00	X	0	0	0
	01	1	1	1	1
	11	1	1	1	1
	10	1	1	1	1

$$A1 = Y3 + Y2$$

		Y1 Y0			
		00	01	11	10
Y3 Y2	00	X	0	1	1
	01	0	0	0	0
	11	1	1	1	1
	10	1	1	1	1

$$A0 = Y3 + Y2' Y1$$



Parity Generator

- A parity bit is an extra bit included with a binary message to make the number of 1's either odd or even.
- A circuit that generates the parity bit in the transmitter is called parity generator.
- It is very useful in systems requiring error detection and correction codes.

Even and Odd parity generator

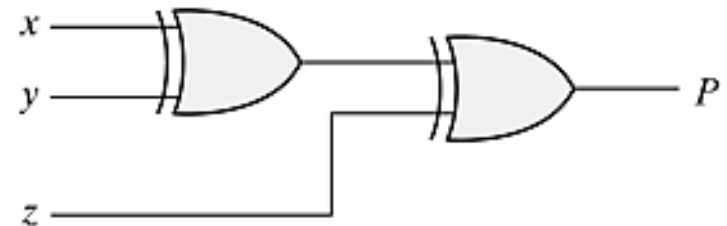
- **Even parity generator** means it **generates “1”** when **input data** has **odd number of “1”** or else it generates “0”
- **Odd parity generator** means it **generates “1”** when **input data** has **even number of “1”** or else it generates “0”.
 - This is complement of Even parity generator

Even Parity Generator

Even-Parity-Generator Truth Table

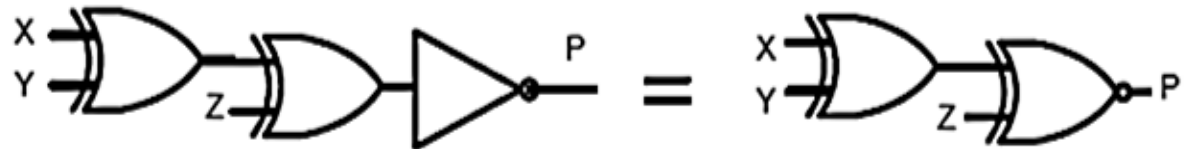
Three-Bit Message			Parity Bit
x	y	z	P
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

$$P = x \oplus y \oplus z$$



Odd Parity Generator

X	Y	Z	P
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	0



$$P = \overline{X \oplus Y \oplus Z}$$

Parity Generator

Message				Parity	
B3	B2	B1	B0	<u>Even</u>	<u>Odd</u>
0	0	0	0	0	1
0	0	0	1	1	0
0	0	1	0	1	0
0	0	1	1	0	1
0	1	0	0	1	0
0	1	0	1	0	1
0	1	1	0	0	1
0	1	1	1	1	0
1	0	0	0	1	0
1	0	0	1	0	1
1	0	1	0	0	1
1	0	1	1	1	0
1	1	0	0	0	1
1	1	0	1	1	0
1	1	1	0	1	0
1	1	1	1	0	1

Even and Odd parity checker

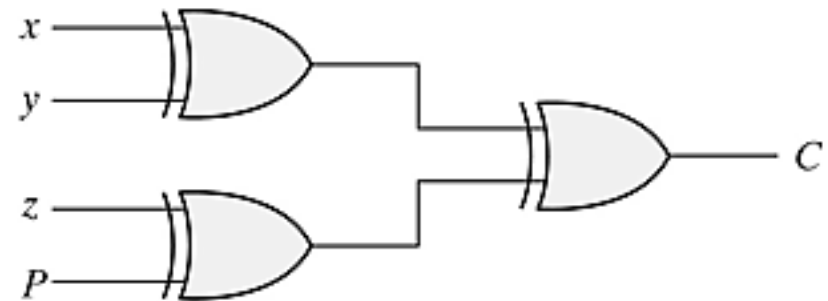
- **Even parity checker** means it generates “1” when input data(Including parity generator bit) has odd number of “1” or else it generates “0”.
- If output is “1”, the data has error (or) it is corrupted data.
- **Odd parity checker** is complement of Even parity checker.

Parity Checker

- The circuit that checks the parity in the receiver is called a *parity checker*.

Even-Parity-Checker Truth Table

Four Bits Received				Parity Error Check
x	y	z	P	C
0	0	0	0	0
0	0	0	1	1
0	0	1	0	1
0	0	1	1	0
0	1	0	0	1
0	1	0	1	0
0	1	1	0	0
0	1	1	1	1
1	0	0	0	1
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	1
1	1	1	0	1
1	1	1	1	0



Odd Parity Checker

4-bit received message				Parity error check C_p
A	B	C	P	
0	0	0	0	1
0	0	0	1	0
0	0	1	0	0
0	0	1	1	1
0	1	0	0	0
0	1	0	1	1
0	1	1	0	1
0	1	1	1	0
1	0	0	0	0
1	0	0	1	1
1	0	1	0	1
1	0	1	1	0
1	1	0	0	1
1	1	0	1	0
1	1	1	0	0
1	1	1	1	1

